

Bayesian physics-informed extreme learning machine for forward and inverse PDE problems with noisy data



Xu Liu^a, Wen Yao^a, Wei Peng^{a,*}, Weien Zhou^a

^a Defense Innovation Institute, Chinese Academy of Military Science, Beijing 100071, China

ARTICLE INFO

Article history:

Received 12 May 2022

Revised 3 September 2022

Accepted 4 June 2023

Available online 13 June 2023

Keywords:

Partial differential equation

Uncertainty quantification

Extreme learning machine

Physics-informed neural network

ABSTRACT

Physics-informed extreme learning machine (PIELM) has recently received significant attention as a rapid version of physics-informed neural network (PINN) for solving partial differential equations (PDEs). The key characteristic is to fix input layer weights with random values and use Moore–Penrose generalized inverse for the output weights. The framework is effective, but it easily suffers from overfitting noisy data and lacks uncertainty quantification for the solution under noise scenarios. To this end, we develop a novel Bayesian physics-informed extreme learning machine (BPIELM) to solve both forward and inverse linear PDE problems with noisy data in a unified framework. In our framework, a prior probability distribution is introduced in the output layer for extreme learning machine with physic laws and the Bayesian method is used to estimate the posterior of parameters. Besides, for inverse PDE problems, problem parameters considered as new output weights are unified in a framework with forward PDE problems. Finally, we demonstrate BPIELM considering both forward problems, including Poisson, advection, and diffusion equations, as well as inverse problems, where unknown problem parameters are estimated. The results show that, compared with PIELM, BPIELM quantifies uncertainty arising from noisy data and provides more accurate predictions. In addition, BPIELM is considerably cheaper than PINN in terms of the computational cost.

© 2023 Elsevier B.V. All rights reserved.

1. Introduction

How to leverage existing data to accurately model and predict PDE-based systems remains an important issue, especially for solving ill-posed problems, e.g., with unknown parameters and boundary conditions, for alleviating the influence by noisy data, and for reducing the computational cost [1,2]. Recently, physics-informed machine learning, as a powerful tool, has attracted increasing attention to addressing those challenges [3]. In particular, physics-informed neural network (PINN), a general framework for solving both forward and inverse PDE problems, encodes underlying physical laws into loss function to constrain the space of admissible solutions and makes predictions for unknown terms given limited data [4–6]. Furthermore, PINN has been successfully used for solving PDEs or complex PDE-based physics problems in various domains, such as materialogy [7], heat conduction [8] and hidden fluid mechanics [9], etc.

Quantifying prediction uncertainty associated with, such as noisy data and unknown terms, is important for neural network (NN) based inference [10,11]. However, PINN without the built-in uncertainty quantification (UQ) may restrict physical and biological applications that require high reliability. For some UQ problems, generating distributions of output predictions is more important than point estimates of the solution [12]. In this regard, Yang et al. [13,14] quantify and propagate uncertainty in systems by combining generative adversarial network (GAN) and physics-based loss function. The latent variable models are constructed as probabilistic representations and the adversarial inference procedure is utilized to train models. While physical laws of systems are encoded into the loss function instead of the discriminator. To fully exploit the adversarial inference procedure, Daw et al. [15] propose a GAN framework based on a physics-informed discriminator, where the physical laws are incorporated into the learning process of both the generator and the discriminator model. In addition, Bayesian neural network (BNN) have the ability to quantify uncertainty by combining scalability, expressiveness, and predictive performance of NNs with the Bayesian learning [16,17]. Yang et al. [18] combine BNN and PINN to provide predictions and quantify uncertainty. Prior probability distributions are

* Corresponding author.

E-mail addresses: liuxu18054448691@126.com (X. Liu), wendy0782@126.com (W. Yao), weipeng0098@126.com (W. Peng).

placed over weights and the inference is then performed on weights, while the inference is not tractable in general. Therefore, variational inference is often used to approximate the inference, but these models require more parameters and more computational costs to converge. To this end, dropout is utilized to estimate the uncertainty in [19], where arbitrary polynomial chaos is combined with PINN to solve stochastic PDE problems. In current researches, GAN, BNN or dropout is integrated with PINN to quantify prediction uncertainty, where PINN is the basic network structure. Besides, PINN is particularly promising for ill-posed forward and inverse problems, while traditional numerical methods, such as finite element methods [20] and finite difference methods [21], require well-defined initial and boundary conditions, which are unavailable in most applications. However, PINN is much slower than traditional numerical methods [22]. Therefore, how to combine the advantages of PINN to achieve numerical equivalent computational costs for quantification uncertainty is a critical challenge for solving PDE-based problems.

Recently, a novel NN called extreme learning machine (ELM) has received great attention owing to its low computational cost and simplicity [23,24]. The most peculiar property of ELM is that its input layer weights are pre-set random values and fixed throughout the training process, and output weights are learned by Moore–Penrose generalized inverse. Inspired by ELM and PINN, Dwivedi et al. [25,26] develop an ELM-based PDE solver called physic-informed extreme learning machine (PIELM) as a rapid version of PINN to solve the linear PDE problems efficiently. The PDE problem is transformed into a linear least squares problem by incorporating physical laws into ELM as the cost function. Subsequently, input layer weights are set randomly from a range according to the neuron number, which is effective to solve elliptic PDEs with sharp gradients [27]. Besides, for nonlinear PDE problems, ELM-based PDE solvers transform the PDE problems into nonlinear least squares problems [28,29], where the NN is also trained by a nonlinear least squares computation, not by gradients-based algorithms. The ELM-based PDE solver as a data-driven technology inevitably faces noise scenarios in real-life applications. The ELM-based PDE solver, however, has two evident drawbacks for ill-posed problems with noisy data:

1. For linear PDE problems, the output weights solved by Moore–Penrose generalized inverse easily suffer from overfitting noisy data and the accuracy of the solver is drastically sensitive to the number of hidden neurons under noise scenarios.
2. The ELM-based PDE solvers are not equipped with the built-in uncertainty quantification at current works, which may restrict their real applications.

BNN provides a natural approach for quantifying uncertainty and is promising for ELM-based PDE solvers, but the main problem of BNN is that the architecture with a large number of successive layers makes it quite redundant, and costly, to account for uncertainty [30]. To this end, a simplified BNN called Bayesian last layer (BLL) considers introducing a distribution function only for the last layer, which utilizes the Bayesian linear model and the finite space represented by the NN to quantify uncertainty [31,32]. Bayesian extreme machine learning (BELM) combines the advantages of BLL and ELM to quantify uncertainty for regression tasks, where a prior distribution is introduced in the output layer of ELM and the Bayesian method is used to estimate posterior distribution [33,34]. Besides, BELM estimates probability distributions of output values, and hence overfitting problem of ELM is solved. Inspired by BELM and PIELM, we propose a novel Bayesian physics-informed extreme learning machine (BPIELM) to solve both forward and inverse PDE problems with noisy data, see Fig. 1, where the Bayesian method is used to quantify the uncertainty from the sparse noisy data. BPIELM consists of two parts: the prior for the extreme learning machine with physical laws and the Bayesian method for estimating posterior distributions of parameters. The first part is to incorporate physical laws into ELM as the cost function and introduce a prior distribution in output weights, then the PDE problem is transformed into a linear least squares problem. In particular, for some inverse PDE problems, the problem parameters are considered as new output weights. The second part is to estimate posterior distributions of parameters and quantify prediction uncertainty for the solution. We demonstrate BPIELM over a range of forward as well as inverse PDE problems and conduct a systematic comparison with PIELM and PINN. In summary, the followings are the main benefits of BPIELM,

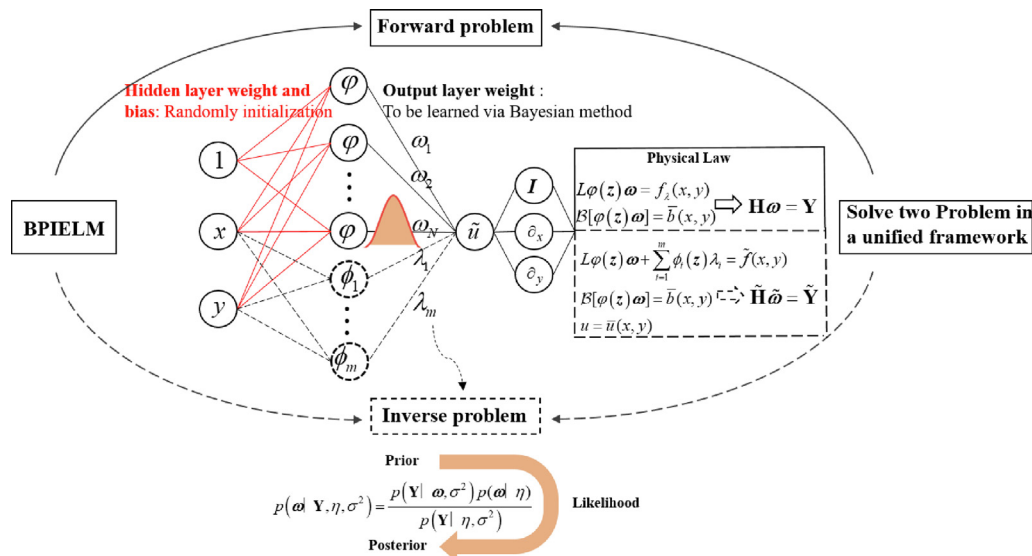


Fig. 1. Conceptual flow of BPIELM with noisy data. $p(\omega|\eta)$ is the prior for output weights as well as problem parameters, $p(Y|\omega, \sigma^2)$ represents the likelihood of measurements, and $p(\omega|Y, \eta, \sigma^2)$ represents the posterior distribution..

1. BPIELM is equipped with the built-in uncertainty quantification, which provides more accurate predictions than PIELM under noise scenarios and has lower computational costs than PINN.
2. Compared to PIELM, BPIELM avoids overfitting by estimating the probability distribution of output values instead of fitting noisy data and hence is not drastically sensitive to the number of hidden neurons.
3. BPIELM is also used for some inverse problems in a unified framework, which achieves a competitive prediction accuracy with PIELM and PINN under noise scenarios.

The rest of the paper is structured as follows. Problem statement is presented in Section 2. In Section 3, we first briefly review PINN and PIELM, and then present BPIELM in detail. Numerical analysis is conducted in Section 4, where we discuss the results of BPIELM, PIELM, and PINN for forward and inverse PDE problems. The conclusions are in Section 5.

2. Problem setup

Suppose we have a linear PDE describing a physical system as follows:

$$\begin{aligned} Lu &= f_\lambda(x, y), (x, y) \in \Omega, \\ \mathcal{B}[u(x, y)] &= b(x, y), (x, y) \in \partial\Omega, \end{aligned} \quad (1)$$

where L is a linear differential operator, $\mathcal{B}$ is a boundary/initial condition operator acting on the boundary $\partial\Omega$, while $u(x, y)$ and $f_\lambda(x, y)$ are prescribed the solution and source terms parameterized by the problem parameter λ , respectively.

In this work, we consider two types of data-driven problems, as summarized in Table 1 and delineated below (refer to [1] for more problems). The first scenario pertains to a forward PDE problem, where L and $\mathcal{B}$ are known. The problem parameter λ is also known and the solution $u(x, y)$ is unknown. Given noisy data of b sampled on $\partial\Omega$, our quantity of interest is to infer the solution $u(x, y)$ at every $(x, y) \in \Omega$ and quantify its uncertainty. Assume we only have N_b sensors on the boundary (including sensors on the initial condition for simplification) and the dataset in this scenario is described as $\mathcal{D} = \{\mathcal{D}_b\}$, $\mathcal{D}_b = \left\{ \left(x_b^{(i)}, y_b^{(i)} \right), \bar{b}^{(i)} \right\}_{i=1}^{N_b}$. Assume that measurements

are independently Gaussian distributed centered at the real value as follows:

$$\bar{b}^{(i)} = b\left(x_b^{(i)}, y_b^{(i)}\right) + \epsilon_b^{(i)}, \quad i = 1, 2 \dots N_b, \quad (2)$$

where $\epsilon_b^{(i)}$ is independent Gaussian noises with zero mean. Assume that the fidelity of each sensor is known and the standard deviation of $\epsilon_b^{(i)}$ is known to be $\sigma_b^{(i)}$.

The second scenario pertains to an inverse PDE problem, where L and $\mathcal{B}$ are known. The problem parameter λ is unknown, while $u(x, y)$ is partially known and some extra measurements on $u(x, y)$ are available. Given noisy data of u in Ω as well as b , our quantity of interest is to infer not only the solution $u(x, y)$ at every $(x, y) \in \Omega$ but the problem parameter λ , and quantify its uncertainty. Assume we have N_u sensors in Ω and N_b sensors on $\partial\Omega$. The dataset in this scenario is described as $\mathcal{D} = \{\mathcal{D}_u, \mathcal{D}_b\}$, where $\mathcal{D}_u = \left\{ \left(x_u^{(i)}, y_u^{(i)} \right), \bar{u}^{(i)} \right\}_{i=1}^{N_u}$ and $\mathcal{D}_b = \left\{ \left(x_b^{(i)}, y_b^{(i)} \right), \bar{b}^{(i)} \right\}_{i=1}^{N_b}$. Similarly, the measurements from $\mathcal{D}_u$ and $\mathcal{D}_b$ are represented as

$$\begin{aligned} \bar{u}^{(i)} &= u\left(x_u^{(i)}, y_u^{(i)}\right) + \epsilon_u^{(i)}, i = 1, 2 \dots N_u, \\ \bar{b}^{(i)} &= b\left(x_b^{(i)}, y_b^{(i)}\right) + \epsilon_b^{(i)}, i = 1, 2 \dots N_b, \end{aligned} \quad (3)$$

where $\epsilon_u^{(i)}$ is also the independent Gaussian noise with zero mean. Assume that the fidelity of each sensor is known and the standard deviation of $\epsilon_u^{(i)}$ is known to be $\sigma_u^{(i)}$.

3. Method

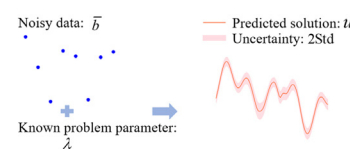
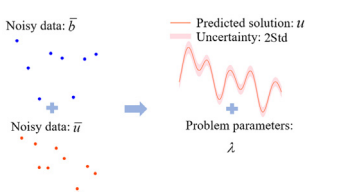
3.1. A primer in physics-informed neural network and physics-informed extreme learning machine

PIELM is considered as a rapid version of PINN, while BPIELM is the combination of PIELM and the Bayesian methods. Therefore, in this subsection, we briefly review PINN and PIELM.

3.1.1. Physics-informed neural network

PINN proposed by Raissi et al. [5] is utilized to solve forward and inverse PDE problems. The key idea of PINN is to encode physical laws into loss function by adding a penalty term to constrain the space of admissible solutions, and then the problem of solving PDEs

Table 1
Problem considered in this paper.

Considered data-driven problem corresponding to following equation $Lu = f_\lambda(x, y), (x, y) \in \Omega$, and $\mathcal{B}[u(x, y)] = b(x, y), (x, y) \in \partial\Omega$		
Forward PDE problem		
$L, \mathcal{B}$	known	
b	noisy data	
$u(x, y)$	unknown	
λ	known	
Dataset	$\mathcal{D} = \{\mathcal{D}_b\}$, where $\mathcal{D}_b = \left\{ \left(x_b^{(i)}, y_b^{(i)} \right), \bar{b}^{(i)} \right\}_{i=1}^{N_b}$	
Objective	$u(x, y)$, uncertainty	
Inverse PDE problem		
$L, \mathcal{B}$	known	
b	noisy data	
$u(x, y)$	partially known	
λ	unknown	
Dataset	$\mathcal{D} = \{\mathcal{D}_u, \mathcal{D}_b\}$, where $\mathcal{D}_u = \left\{ \left(x_u^{(i)}, y_u^{(i)} \right), \bar{u}^{(i)} \right\}_{i=1}^{N_u}$, $\mathcal{D}_b = \left\{ \left(x_b^{(i)}, y_b^{(i)} \right), \bar{b}^{(i)} \right\}_{i=1}^{N_b}$	
Objective	$\lambda, u(x, y)$, uncertainty	

in Eq. (1) is transformed into an optimization problem of minimizing the loss function. Concretely, a multi-layer perception with the activation function is first constructed. Then, to measure the difference between the NN and physical laws, the loss function is defined as

$$\mathcal{L}(\theta) = w_f \mathcal{L}_f(\theta; \mathcal{T}_f) + w_b \mathcal{L}_b(\theta; \mathcal{T}_b) + w_u \mathcal{L}_u(\theta; \mathcal{T}_u), \quad (4)$$

where

$$\begin{aligned} \mathcal{L}_f(\theta; \mathcal{T}_f) &= \frac{1}{|\mathcal{T}_f|} \sum_{\mathbf{x} \in \mathcal{T}_f} \|f(t, \mathbf{x})\|^2, \\ \mathcal{L}_b(\theta; \mathcal{T}_b) &= \frac{1}{|\mathcal{T}_b|} \sum_{\mathbf{x} \in \mathcal{T}_b} \|\mathcal{B}(\mathbf{x}) - b\|^2, \\ \mathcal{L}_u(\theta; \mathcal{T}_u) &= \frac{1}{|\mathcal{T}_u|} \sum_{\mathbf{x} \in \mathcal{T}_u} \|\hat{u}(\mathbf{x}) - u(\mathbf{x})\|^2. \end{aligned} \quad (5)$$

w_f, w_b as well as w_u are hyper-parameters of weights. f represents the PDE residual in Eq. (1). θ represents weights and bias of the NN. $\hat{u}$ denotes the output of the NN. $\mathcal{T}_f, \mathcal{T}_b$, and $\mathcal{T}_u$ denote training points from PDE residual, boundary, and data constraints, respectively. In other words, the loss function of PINN is a weighted sum of multiple terms, including PDE, boundary, and data constraints.

Finally, the NN is trained to search for the best parameters θ by minimizing the loss function, where gradient-based algorithms are used to minimize the loss function.

3.1.2. Physics-informed extreme learning machine

Dwivedi et al. [25] propose a rapid version of PINN called PIELM, which is applied to solve stationary and time-dependent linear PDEs. The most peculiar property is that PIELM includes a single-layer feed-forward NN, where its input layer weights are fixed with random values and only output weights are training parameters. Then, physical laws are incorporated into the NN as the cost function. In particular, the linear nature of PDEs and the physical laws in Eq. (1) are combined to solve a linear equation,

$$\mathbf{H}\mathbf{c} = \mathbf{Y}, \quad (6)$$

where $\mathbf{c}$ is the unknown output weights and $\mathbf{H}$ as well as $\mathbf{Y}$ are known matrices that contain physical laws (PDE, boundary, and data constraints). Finally, the prediction of $\mathbf{c}$ comes from the linear least squares solution by

$$\mathbf{c}_{pred} = \mathbf{H}^\dagger \mathbf{Y}, \quad (7)$$

where $\mathbf{H}^\dagger = (\mathbf{H}^T \mathbf{H})^{-1} \mathbf{H}^T$ represents the Moore–Penrose generalized inverse matrix of $\mathbf{H}$.

3.2. Bayesian physics-informed extreme learning machine

BPIELM is an ELM-based PDE solver that combines two ideas from PIELM and BELM. The first idea, coming from PIELM, is to incorporate physical laws into ELM as the cost function, transforming linear PDE problems into a linear least squares problem. The other one, coming from Bayesian ELM [33,34], is combining the Bayesian method with ELM to naturally quantify uncertainty for regression and classification tasks. The output weights are defined as a prior distribution and are optimized by the Bayesian method, not by Moore–Penrose generalized inverse. Since PIELM is not equipped with built-in uncertainty quantification and is prone to overfit noisy data, we resolve two drawbacks by combining PIELM with the Bayesian framework. In addition, we also unify the forward and inverse PDE problems in one framework. BPIELM consists of two parts: the prior for extreme learning machine with physical laws and the Bayesian method for estimating posterior distributions of parameters. The first part is incorporating physical laws into the NN as the cost function and introducing a prior in the out-

put weights, then the PDE problem is transformed into a linear least squares problem. The second part is to estimate posterior distributions of parameters and quantify prediction uncertainty. Fig. 1 shows the conceptual flow of BPIELM and the detailed descriptions are provided in Algorithm 1.

Algorithm 1: Bayesian physics-informed extreme machine for forward and inverse PDE problems

Input:

$\bar{b}$, noisy measurements on the boundary $\partial\Omega$.

$\bar{u}$, noisy measurements in the domain Ω .

Output:

$u(x, y)$ and uncertainty (λ for inverse problems).

1// 2 The prior for extreme learning machine with physical laws

3 Construct a single-layer NN with N neurons and initialize input layer weights randomly.

4 Define the training errors, ξ_f, ξ_b (ξ_u for inverse problems).

5 Transform the linear system into a linear equation and the system for forward or inverse problems is written as $\mathbf{H}\omega = \mathbf{Y}$ for simplifying representation.

6 Suppose the prior distribution of parameters $p(\omega|\eta)$ follows a Gaussian distribution with the hyper-parameter η .

7// The Bayesian method for estimating posterior distributions of parameters

8 Define the likelihood function $p(\mathbf{Y}|\omega, \sigma^2)$ with the hyper-parameter σ .

9 Construct the posterior by the prior distribution and the likelihood function,

$$p(\omega|\mathbf{Y}, \eta, \sigma^2) = \frac{p(\mathbf{Y}|\omega, \sigma^2)p(\omega|\eta)}{p(\mathbf{Y}|\eta, \sigma^2)}.$$

10 Use Evidence Procedure to optimize hyper-parameters, η and σ .

11 Calculate output $\mathbf{Y}_{new}$ by the integral of the posterior distribution for input instances $\mathbf{x}_{new}$ and obtain λ by the new output weights for inverse problems.

12 Obtain the uncertainty by the variance,

$$\sigma^2(\mathbf{x}_{new}) = \sigma^2 + \mathbf{x}_{new}^T \Sigma \mathbf{x}_{new}, \quad \Sigma = (\eta \mathbf{I} + \sigma^{-2} \mathbf{H}^T \mathbf{H})^{-1}.$$

3.2.1. The prior for extreme learning machine with physical laws

We consider a single-layer NN with N neurons in Fig. 1. The key of ELM is to pre-set the input layer weights randomly, which are fixed throughout the training process. The parameters of the input layer are defined as $\mathbf{x} = [x, y, 1]^T$, $\alpha = [\alpha_1, \alpha_2, \dots, \alpha_n]^T$, $\beta = [\beta_1, \beta_2, \dots, \beta_n]^T$, $\gamma = [\gamma_1, \gamma_2, \dots, \gamma_n]^T$, where α, β and γ are input layer weights. The nonlinear activation function is $\varphi = \tanh$ and output weights are $\omega = [\omega_1, \omega_2, \dots, \omega_n]^T$. Then the output of k^{th} hidden neuron is represented by $\varphi(z_k)$, where $z_k = [\alpha_k, \beta_k, \gamma_k] \mathbf{x}$. The output of the NN is written as

$$\hat{u}(\mathbf{x}) = \varphi(\mathbf{z})\omega. \quad (8)$$

Given $\hat{u}(\mathbf{x})$, the partial derivatives of Eq. (8), with respect to independent variables, are

$$\begin{aligned} \frac{\partial^k \hat{u}}{\partial x^k} &= \sum_{j=1}^n \omega_j \alpha_j^k \frac{\partial^k \varphi}{\partial z^k}, \\ \frac{\partial^k \hat{u}}{\partial y^k} &= \sum_{j=1}^n \omega_j \beta_j^k \frac{\partial^k \varphi}{\partial z^k}, \end{aligned} \quad (9)$$

where k refers to the k^{th} order derivative.

Therefore, according to Eq. (1)–(3), physical laws are incorporated into the NN by introducing physics-based training errors, ξ_f, ξ_b . The training errors ξ_f, ξ_b are defined as errors in approximating PDE and boundary/initial condition,

$$\begin{aligned}\xi_f &= L\varphi(\mathbf{z})\boldsymbol{\omega} - f_\lambda(x, y) = 0, \\ \xi_b &= B[\varphi(\mathbf{z})\boldsymbol{\omega}] - \bar{b}(x, y) = 0.\end{aligned}\quad (10)$$

For the forward problems, Eq. (10) leads to a linear equation,

$$\mathbf{H}\boldsymbol{\omega} = \mathbf{Y}, \quad (11)$$

where $\mathbf{H} \in \mathbb{R}^{N^* \times N}$, $N^* = N_f + N_b$, N is equal to the neuron number. $\mathbf{H}$ and $\mathbf{Y}$ are matrices determined by L and B . The output weight $\boldsymbol{\omega}$ is the training parameters.

In particular, for some inverse PDE problems in this work, we consider the type of PDEs, and its training errors approximating PDEs can be written by separating variables as

$$\xi_f = L\varphi(\mathbf{z})\boldsymbol{\omega} + \sum_{j=1}^m \phi_j(\mathbf{z})\lambda_j - \tilde{f}(x, y) = 0, \quad (12)$$

where ϕ_j and $\tilde{f}$ are determined by separating variables. Under noisy data in Eq. (3), the training errors approximating PDE, boundary/initial condition, and data constraints are represented by

$$\begin{aligned}\xi_f &= (H_f^1, H_f^2) \begin{pmatrix} \boldsymbol{\omega} \\ \lambda \end{pmatrix} - \tilde{f}(x, y) = 0, \\ \xi_b &= (H_b, 0) \begin{pmatrix} \boldsymbol{\omega} \\ \lambda \end{pmatrix} - \bar{b}(x, y) = 0, \\ \xi_u &= (H_u, 0) \begin{pmatrix} \boldsymbol{\omega} \\ \lambda \end{pmatrix} - \bar{u}(x, y) = 0.\end{aligned}\quad (13)$$

For the inverse problems, Eq. (12) and (13) also lead to a linear system,

$$\tilde{\mathbf{H}}\tilde{\boldsymbol{\omega}} = \tilde{\mathbf{Y}}, \quad (14)$$

where $\tilde{\boldsymbol{\omega}} = [\boldsymbol{\omega}, \lambda]^T$, $\tilde{\mathbf{H}} \in \mathbb{R}^{N^* \times N}$, $N^* = N_f + N_b + N_u$ and $\tilde{\mathbf{Y}}$ are matrices determined by L and B . The problem parameters are also considered as output weights.

In summary, physical laws are encoded into the linear system by using the linear nature of linear PDEs. To simplify the following representation, the linear system for forward or inverse problems is written as $\mathbf{H}\boldsymbol{\omega} = \mathbf{Y}$, where $\boldsymbol{\omega}$ is the training parameters. In most applications of Bayesian theory, a commonly used prior for $\boldsymbol{\omega}$ is that each component of $\boldsymbol{\omega}$ follows an independent Gaussian distribution with zero mean,

$$p(\boldsymbol{\omega}|\eta) = \mathcal{N}(\mathbf{0}; \eta^{-1}\mathbf{I}), \quad (15)$$

where $\mathbf{I}$ is the identity matrix and η is a hyperparameter, which will be optimized the following steps.

3.2.2. The Bayesian method for estimating posterior distributions of parameters

In this subsection, the Bayesian method is used to estimate posterior distributions of parameters, improving the over-fitting problem of ELM-based solvers and quantifying prediction uncertainty. According to the wide application of Bayesian theory, most models are divided into two steps:

1. The posterior distribution is proportional to the product of the prior distribution and the likelihood function,

$$p(\boldsymbol{\omega}|\mathbf{D}) \propto p(\boldsymbol{\omega})p(\mathbf{D}|\boldsymbol{\omega}), \quad (16)$$

where $\boldsymbol{\omega}$ and $\mathbf{D}$ represent the set of parameters and the dataset. In this work, variational inference is used to obtain the posterior dis-

tribution $p(\boldsymbol{\omega}|\mathbf{D})$ by optimizing over a family of tractable distribution. Concretely, we introduce a prior distribution in output weights and the likelihood function with the hyper-parameters. Then Evidence Procedure [35] is used to optimize hyper-parameters for estimating posterior distribution.

2. The output $\mathbf{Y}_{\text{new}}$ can be given by the integral of the posterior distribution of $\boldsymbol{\omega}$ for input instances of $\boldsymbol{\chi}_{\text{new}}$,

$$p(\mathbf{Y}_{\text{new}}|\boldsymbol{\chi}_{\text{new}}, \mathbf{D}) = \int p(\mathbf{Y}_{\text{new}}|\boldsymbol{\chi}_{\text{new}}, \boldsymbol{\omega})p(\boldsymbol{\omega}|\mathbf{D})d\boldsymbol{\omega}. \quad (17)$$

Generally, the integral of Eq. (17) is analytically intractable and Monte Carlo Approximation [36] is therefore typically applied in this work. To give a posterior at any $\boldsymbol{\chi}$, we can sample from $p(\boldsymbol{\omega}|\mathbf{D})$, denoted as $\{\boldsymbol{\omega}^{(i)}\}_{i=1}^M$, and then obtain statistics from samples $\{\mathbf{Y}(\boldsymbol{\chi}; \boldsymbol{\omega}^{(i)})\}_{i=1}^M$. We focus mostly on the mean and standard deviation of $\{\mathbf{Y}(\boldsymbol{\chi}; \boldsymbol{\omega}^{(i)})\}_{i=1}^M$, since the former represents the prediction of $\mathbf{Y}$ while the latter quantifies the uncertainty [37,38].

Suppose the linear system follows the relationship,

$$\mathbf{Y} = \mathbf{H}\boldsymbol{\omega} + \epsilon, \quad (18)$$

where ϵ follows a Gaussian distribution $\mathcal{N}(0; \sigma^2)$. The likelihood function is then derived as

$$p(\mathbf{Y}|\boldsymbol{\omega}, \sigma^2) = \left(\frac{1}{2\pi\sigma^2}\right)^{\frac{N^*}{2}} \exp\left(-\frac{\|\mathbf{Y} - \mathbf{H}\boldsymbol{\omega}\|^2}{\sigma^2}\right), \quad (19)$$

where N^* is the dimension number of $\mathbf{H}$.

Under the assumption that the prior distribution and the likelihood function follow Gaussian distributions, the posterior can be derived as

$$p(\boldsymbol{\omega}|\mathbf{Y}, \eta, \sigma^2) = \frac{p(\mathbf{Y}|\boldsymbol{\omega}, \sigma^2)p(\boldsymbol{\omega}|\eta)}{p(\mathbf{Y}|\eta, \sigma^2)}, \quad (20)$$

which also follows a Gaussian distribution with a mean value of $\boldsymbol{\mu}$ and a variance of $\boldsymbol{\Sigma}$ [39],

$$\begin{aligned}\boldsymbol{\mu} &= \sigma^{-2}\boldsymbol{\Sigma}\mathbf{H}^T\mathbf{Y}, \\ \boldsymbol{\Sigma} &= (\eta\mathbf{I} + \sigma^{-2}\mathbf{H}^T\mathbf{H})^{-1}.\end{aligned}\quad (21)$$

Then the optimal values of hyper-parameters, η and σ , are obtained by Evidence Procedure [40]. The optimal conditions are iteratively calculated by

$$\begin{aligned}\eta &\leftarrow \frac{N - \eta \cdot \text{trace}[\boldsymbol{\mu}]}{\boldsymbol{\Sigma}^T\boldsymbol{\Sigma}}, \\ \sigma^2 &\leftarrow \frac{\|\mathbf{Y} - \mathbf{H}\boldsymbol{\omega}\|^2}{N^* - N - \eta \cdot \text{trace}[\boldsymbol{\mu}]},\end{aligned}\quad (22)$$

where N^* is the dimension number of $\mathbf{H}$ and N is the number of output weights. The iterative process stops when $\boldsymbol{\mu}$ falls below a given threshold.

Finally, given a $\boldsymbol{\chi}_{\text{new}}$, Monte Carlo Approximation is used to obtain the output $\mathbf{Y}_{\text{new}}$ by the posterior distribution. Besides, the output follows a distribution,

$$p(\mathbf{Y}_{\text{new}}|\mathbf{Y}, \eta, \sigma^2) = \mathcal{N}(\mathbf{H}_{\text{new}}\boldsymbol{\omega}; \sigma^2(\boldsymbol{\chi}_{\text{new}})), \quad (23)$$

where $\mathbf{H}_{\text{new}} = \varphi([\boldsymbol{\alpha}, \boldsymbol{\beta}, \boldsymbol{\gamma}]\boldsymbol{\chi}_{\text{new}})$ and the variance is calculated by

$$\sigma^2(\boldsymbol{\chi}_{\text{new}}) = \sigma^2 + \boldsymbol{\chi}_{\text{new}}^T \boldsymbol{\Sigma} \boldsymbol{\chi}_{\text{new}}. \quad (24)$$

4. Numerical result

In this section, we will first consider solving forward and inverse PDE problems by BPIELM over PIELM and PINN. The forward PDE problem including Poisson, advection and diffusion

equations. The inverse PDE problem includes Poisson and Helmholtz equations, where noisy data is used to solve the solution and identify the unknown problem parameters. All the experiments are conducted the very common machine with a 2.8-GHz Intel(R) Xeon(R) Gold 6242 CPU, 128-GB memory, and NVIDIA GeForce RTX 3090 GPU.

4.1. Poisson equation

The two dimensional (2D) Poisson equation is given by

$$u_{xx} + u_{yy} = (16x^2 + 64y^2 - 12)e^{-(2x^2+4y^2)}, (x, y) \in \Omega, \quad (25)$$

where the solution is $u = \frac{1}{2} + e^{-(2x^2+4y^2)}$, $\Omega = \{(x, y) | x = 0.55\rho(\theta)\cos(\theta), y = 0.75\rho(\theta)\sin(\theta)\}$, and $\rho(\theta) = 1 + \cos(\theta)\sin(4\theta)$, $0 \leq \theta \leq 2\pi$. Assuming that the boundary condition is unknown, we consider the Gaussian noise in the measurements, and only have N_b sensors on the boundary, which are equidistantly distributed.

To solve the Poisson equation in the BPIELM framework, we encode the governing equation in Eq. (25) and sensor measurements on the boundary into the linear system. Then, we use the Bayesian method to estimate the output weights, where the hyperparameters are $\eta = 2e - 1$ and $\beta = 1$ (the default setting in the following tests). In this case, the neuron number is $N = 100$ and the number of training points is $N_f = 400$ and $N_b = 38$. The noise scale on the boundary is $\epsilon_b \sim \mathcal{N}(0, 0.01^2)$. Fig. 2 shows the results of PIELM and BPIELM, where BPIELM provides more accurate predictions for the solution and gives uncertainty quantification for the solution. In BPIELM, the standard deviation for the case is large on the boundary even though there are some sensors on the boundary. Besides, the area with a large error has a relatively large standard deviation. The mean absolute errors (MAEs) of PIELM and

BPIELM are 0.029 and 0.019, respectively. The areas with large errors are mainly concentrated in the areas without sensors, while BPIELM gives predicted means closer to the exact solutions on the right-wing of the butterfly than PIELM. Fig. 3 provides further comparisons between PIELM and BPIELM under two different noise scales, i.e., $\epsilon_b \sim \mathcal{N}(0, 0.01^2)$ and $\epsilon_b \sim \mathcal{N}(0, 0.1^2)$. BPIELM provides predictions closer to the exact solution than PIELM under two noise scales, and the errors of BPIELM are mostly bounded by two standard deviations, which increase as the noise scale increases. Moreover, the errors between predictions and the exact solution increase with the increasing noise scale. Fig. 4 illustrates the distributions of output weights under noise and noiseless scenarios. In BPIELM, the posterior distributions $p(\omega|\mathbf{Y}, \eta, \sigma^2)$ under noise and noiseless scenarios are close. In contrast, the distribution $p(\omega|\mathbf{Y})$ by PIELM under noise scenarios is quite different from the distribution under noiseless scenarios, causing the worse performance of predictions. It can be seen that BPIELM uses the Bayesian method to learn the distribution of output weights, which is better than the Moore–Penrose generalized inverse by PIELM.

The effect of the different sensor number for PIELM and BPIELM is illustrated in Fig. 5. In the first column, we compare the maximum absolute error on the whole domain (Max-AE) and the MAE of PIELM and BPIELM. BPIELM achieves much better MAEs than PIELM with a smaller sensor number. Besides, BPIELM achieves smaller Max-AEs than PIELM at the same sensor number. This is because the pseudo-inverse operation is used in PIELM so as to be prone to overfit noisy data, causing the abnormal predictions at some points, but BPIELM alleviates overfitting to decrease Max-AEs. Especially, the results of PIELM and BPIELM with 19 sensors are shown in the last two columns. The errors of PIELM on the two wings of the butterfly tend to exceed 9, but BPIELM reduces the error to be less than 0.1. In summary, the above results show

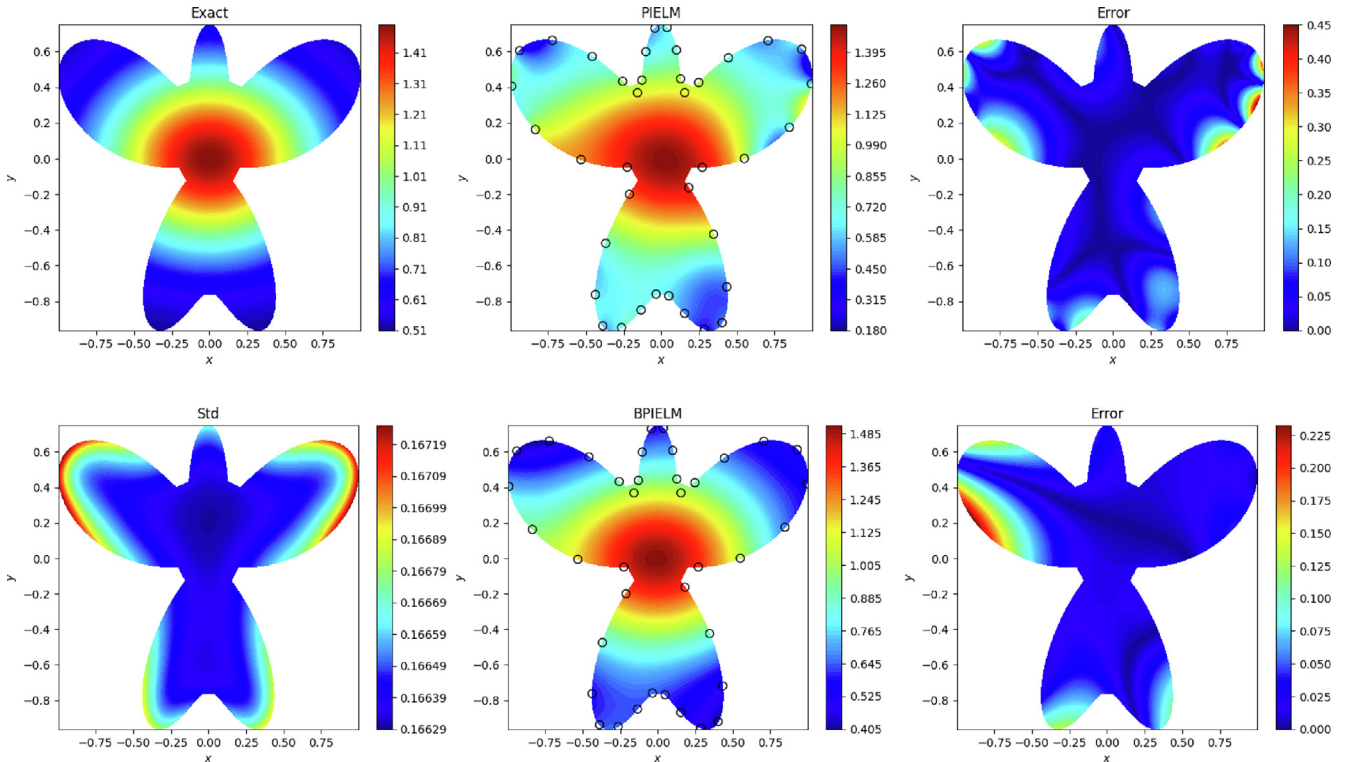


Fig. 2. Poisson equation: The first row represents the exact solution, the PIELM solution and its absolute error, respectively. The second row represents the standard deviations for u using BPIELM, the BPIELM solution and its absolute error, respectively. The noise scale on the boundary is $\epsilon_b \sim \mathcal{N}(0, 0.01^2)$, the neuron number is $N = 100$, and the number of training points is $N_f = 400$ and $N_b = 38$. Black circles represent the positions of sensors.

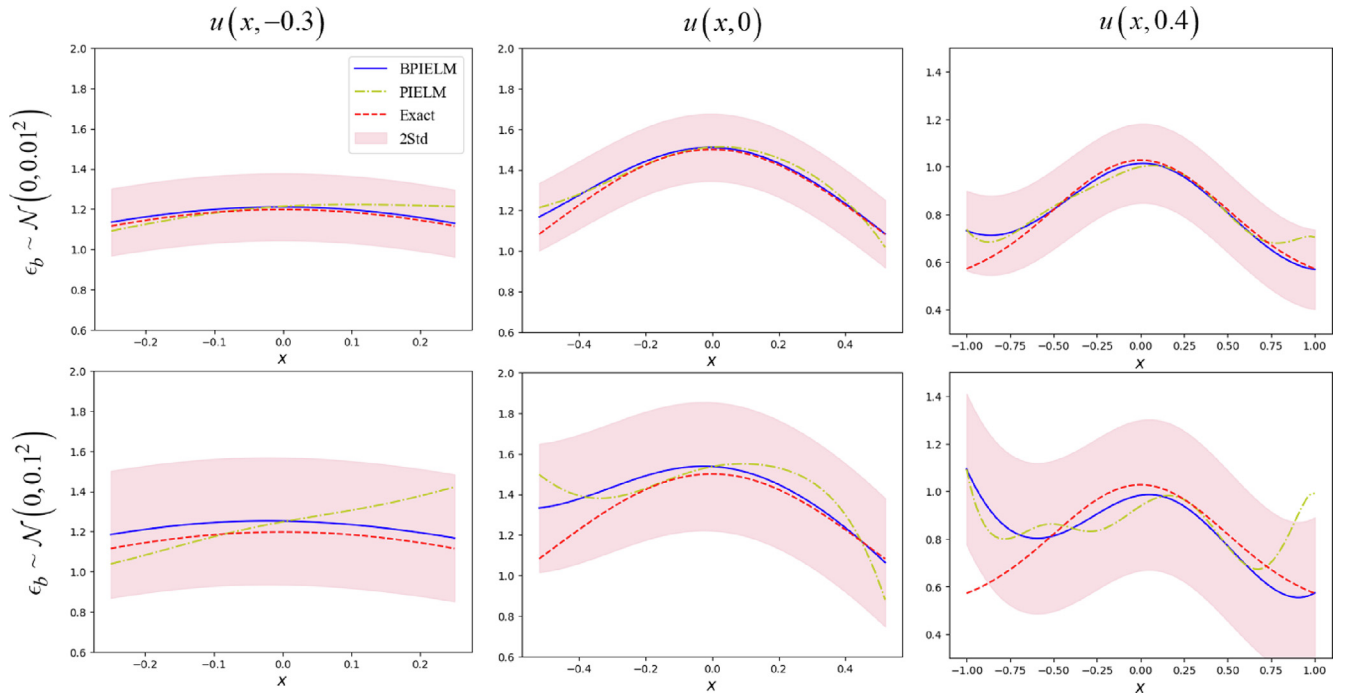


Fig. 3. Poisson equation: The columns (from left to right) represent comparisons between BPIELM and PIELM at $y = -0.3, y = 0$ and $y = 0.4$, where $N = 100, N_f = 400$ and $N_b = 41$ are used. The row (from top to down) represents two noise scales on the boundary, $\epsilon_b \sim \mathcal{N}(0, 0.01^2)$ and $\epsilon_b \sim \mathcal{N}(0, 0.1^2)$.

that the sensor number has a stronger effect for PIELM on the predictive accuracy than BPIELM.

PIELM is unstable for the neuron number and is easily affected by noisy data. To this end, we compare the effect of the different neuron number with a fixed number of training points on the predictive accuracy. The results of PIELM and BPIELM are shown in Fig. 6. Under noise scenarios, PIELM hardly achieves the best MAE when the neuron number is equal to or close to the number of constraints, i.e., $N = N_f + N_b$. The solution accuracy of PIELM is sensitive to the neuron number. Under the small neuron number, MAEs between PIELM and BPIELM are close. It means that a shallow NN architecture both for PIELM and BPIELM can not learn enough physics information to depict the whole solution. As the

neuron number increases, MAEs of PIELM increase after reaching the minimum value in a certain range, while MAEs of BPIELM converge to a small value and fluctuate in a small range. It indicates that an unusually large neuron number in PIELM causes overfitting, but BPIELM alleviates the overfitting issues. Especially for $N > N^*$, BPIELM gives much better predictions than PIELM.

Table 2 is a further comparison of PINN (GPU), PIELM (CPU) and BPIELM (CPU) under three noise scales, i.e., $\epsilon_b \sim \mathcal{N}(0, 0.01^2)$, $\epsilon_b \sim \mathcal{N}(0, 0.05^2)$ and $\epsilon_b \sim \mathcal{N}(0, 0.1^2)$. In terms of the computational cost (the network training time), PINN takes around 140 s to train with Adam [41]. In contrast, the training times of PIELM and BPIELM are on the order of a second and on the order of two

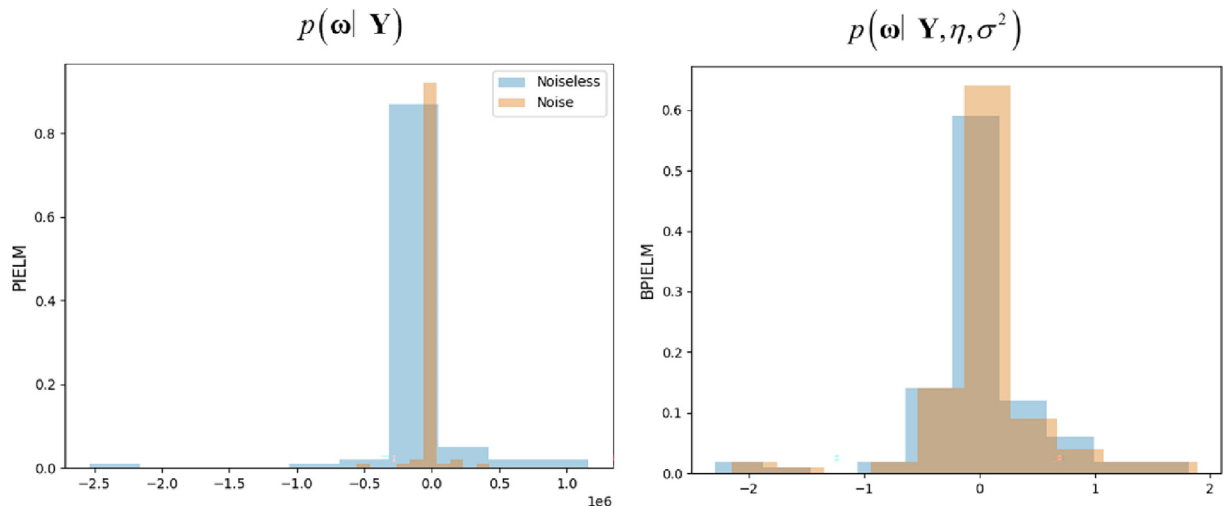


Fig. 4. Poisson equation: The rows (from left to right) represent $p(\omega|Y)$ by PIELM and $p(\omega|Y, \eta, \sigma^2)$ by BPIELM. The noise scale is $\epsilon_b \sim \mathcal{N}(0, 0.01^2)$. The neuron number of and training points is $N = 100, N_f = 400$, and $N_b = 38$, respectively..

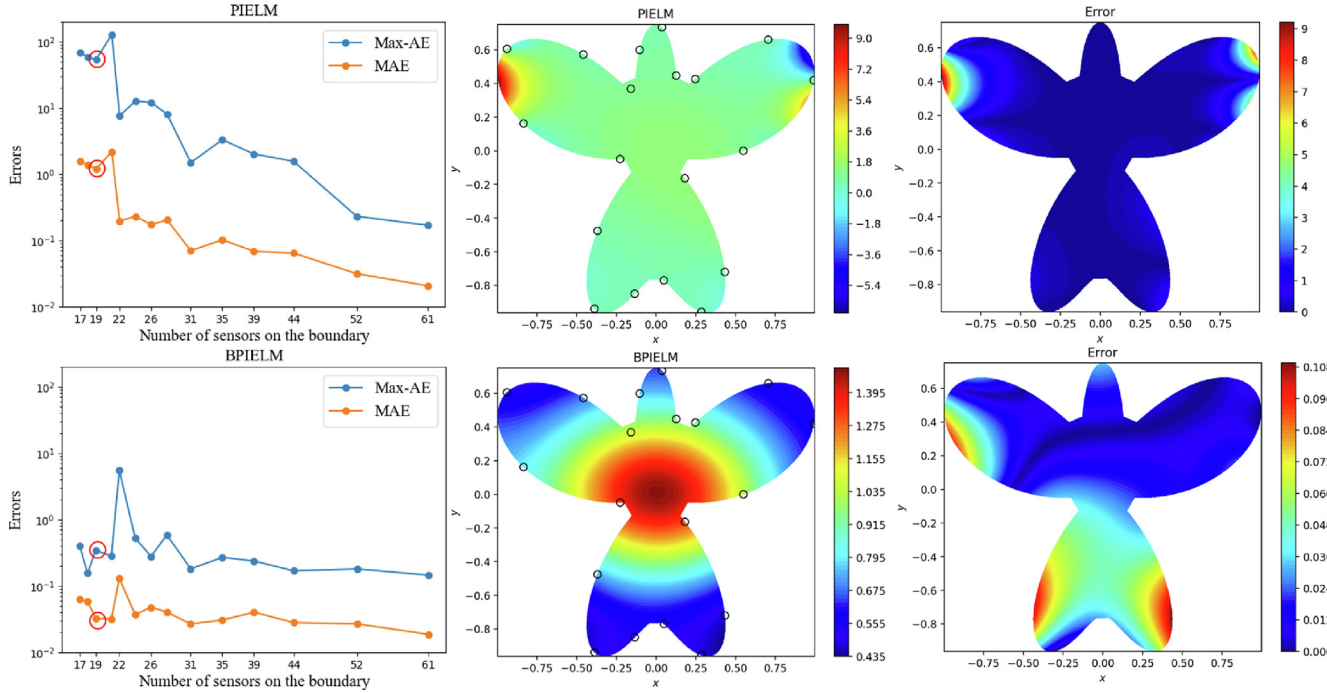


Fig. 5. Poisson equation: The first column represents the Max-AE and the MAE under different sensor number on the boundary, where $\epsilon_b \sim \mathcal{N}(0, 0.01^2)$, $N = 100$ and $N_f = 200$ are used. The second and third columns represent predictions at $N_b = 19$ and its absolute error. The rows (from top to down) represent the results of PIELM and BPIELM. Black circles represent the positions of sensors.

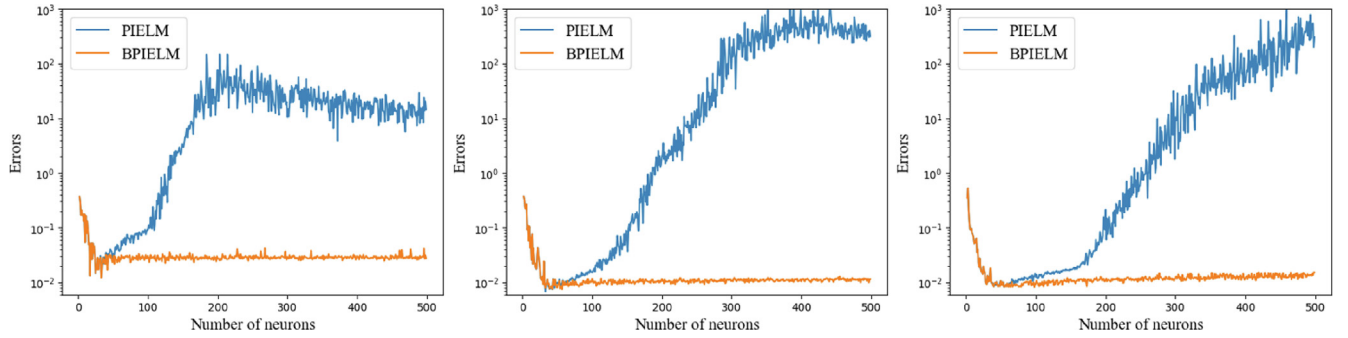


Fig. 6. Poisson equation: The columns (from left to right) represent the MAE comparisons between BPIELM and PIELM at $N_{bc} = 21, N_{bc} = 42$ and $N_{bc} = 63$, where $\epsilon_b \sim \mathcal{N}(0, 0.01^2)$ and $N_f = 400$ are used.

seconds, respectively. For the accuracy of the solution, BPIELM provides more accurate predictions than PIELM. The MAE of BPIELM is better than PINN, while PINN is superior to BPIELM in terms of the Max-AE. In a word, BPIELM is not only much more accurate than PIELM but also considerably cheaper than PINN in terms of computational cost.

4.2. Advection equation

The advection equation in one and two spatial dimensions (plus time) is given by

$$\begin{aligned} \frac{\partial u}{\partial t} - c \frac{\partial u}{\partial x} &= 0, (x, t) \in \Omega, \\ u(x_1, t) &= u(x_2, t), u(x, 0) = h(x), \end{aligned} \quad (26)$$

where $u(x, t)$ is the solution, the constant $c = -2$ is the wave speed, $x_1 = 0$ as well as $x_2 = 1$ are the boundary points, and $\Omega = \{(x, t) | x \in [0, 1], t \in [0, 1]\}$. The exact solution is considered as

$$u(x, t) = 2 \operatorname{sech} \left[3 \left(-\frac{1}{2} + \xi \right) \right], \quad (27)$$

where

$$\xi = \operatorname{mod} \left(x - x_0 + ct + \frac{L}{2}, L \right), \quad L = x_2 - x_1, \quad x_0 = 0.5. \quad (28)$$

Assuming that boundary conditions $(u(x_1, t), u(x_2, t))$ and $h(x)$ are unknown, we consider the Gaussian noise in measurements, and only have N_b sensors, which are equidistantly distributed on the boundary.

To employ BPIELM for solving the advection equation, we encode the governing equations and sensor measurements on the boundary into the linear system, and then use the Bayesian method to estimate output weights. In this simulation, the neuron number is $N = 150$ and the number of training points is $N_f = 400$ and $N_b = 28$. The noise scale on the boundary is $\epsilon_b \sim \mathcal{N}(0, 0.1^2)$. The results of PIELM and BPIELM are illustrated in Fig. 7, where data noise scale on the boundary is $\epsilon_b \sim \mathcal{N}(0, 0.1^2)$. Under the large noise scale, BPIELM provides better predictions for the solution than PIELM and gives uncertainty quantification for the solu-

Table 2

Poisson equation: The comparisons of PINN, PIELM and BPIELM under three noise scales, i.e., $\epsilon_b \sim \mathcal{N}(0, 0.01^2)$, $\epsilon_b \sim \mathcal{N}(0, 0.05^2)$ and $\epsilon_b \sim \mathcal{N}(0, 0.1^2)$. The number of training points is $N_f = 400$, $N_b = 19$ and the number of neurons is $N = 100$. PINN has four hidden layers with 100 neurons in each layer and the epoch is 3,000.

Noise scale	0.01			0.05			0.1		
	Max-AE	MAE	Time/s	Max-AE	MAE	Time/s	Max-AE	MAE	Time/s
PINN	0.165	0.043	142.30	0.251	0.055	141.02	0.327	0.072	143.32
PIELM	0.447	0.029	1.21	0.998	0.065	1.22	1.41	0.092	1.22
BPIELM	0.232	0.019	2.05	0.516	0.047	2.67	0.759	0.067	2.65

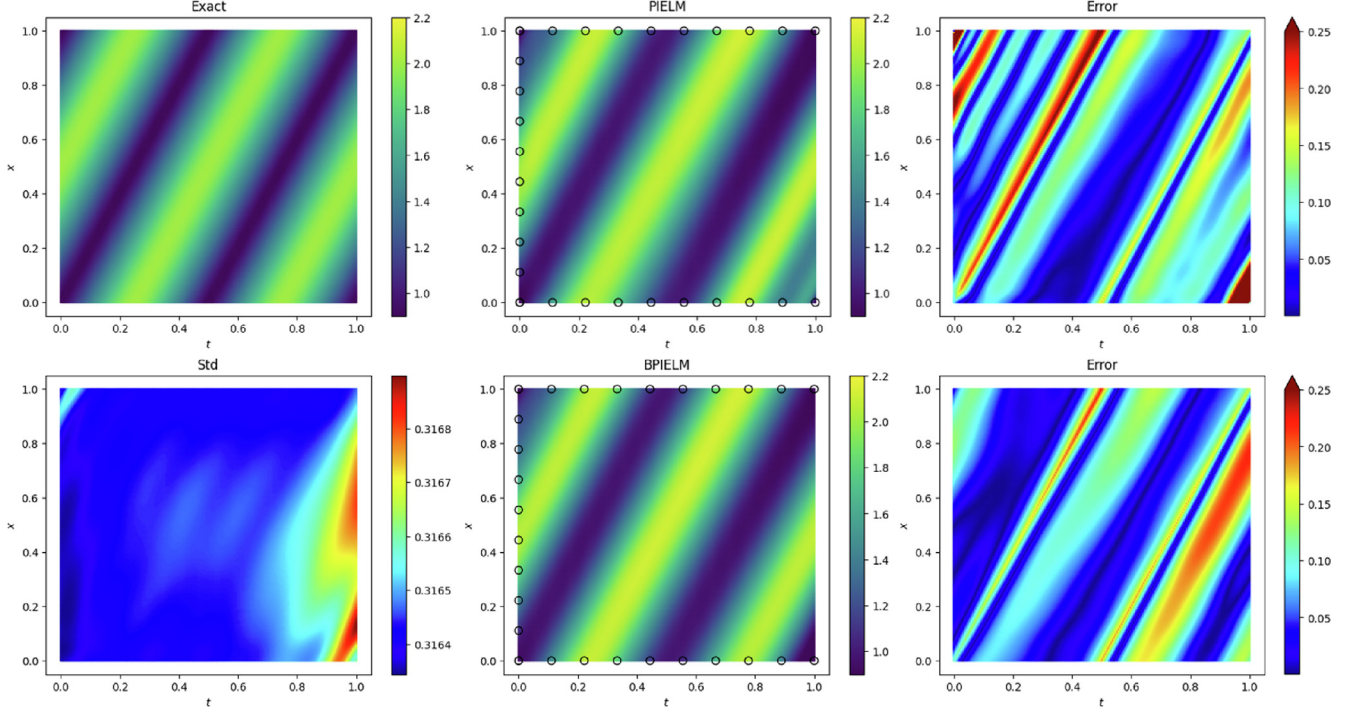


Fig. 7. Advection equation: The first row represents the exact solution, the PIELM solution, and its absolute error, respectively. The second row represents standard deviations for u using BPIELM, the BPIELM solution, and its absolute error. The data noise scale on the boundary is $\epsilon_b \sim \mathcal{N}(0, 0.1^2)$, the number of neurons is $N = 150$ and the number of training points is $N_f = 400$ and $N_b = 28$. Black circles represent the positions of sensors.

tion. Concretely, MAEs of PIELM and BPIELM are 0.070 and 0.066, respectively. For BPIELM, the area with large errors corresponds to relatively large standard deviations. Furthermore, according to three temporal snapshots $t = 0, 0.5, 1$, Fig. 8 provides the comparisons between PIELM and BPIELM under two different noise scales, i.e., $\epsilon_b \sim \mathcal{N}(0, 0.01^2)$ and $\epsilon_b \sim \mathcal{N}(0, 0.1^2)$. The errors of BPIELM are mostly bounded by two standard deviations, and the errors between predictions and the exact solution increase with the increasing noise scale. Under the small noise scale, BPIELM and PIELM provide predictions close to the exact solution. When the noise scale is large, BPIELM gives much better predictions than PIELM, especially for the area near the boundary. Fig. 9 shows the distributions of output weights noise and noiseless scenarios. Obviously, in terms of $p(\omega|\mathbf{Y})$, PIELM under noise scenarios uses Moore–Penrose generalized inverse to learn output weights, which are quite different from the weights under noiseless scenarios. However, in BPIELM, the posterior distributions $p(\omega|\mathbf{Y}, \eta, \sigma^2)$ under noise and noiseless scenarios are close.

Fig. 10 illustrates the effect of the different sensor number for PIELM and BPIELM. In the first column, we compare Max-AEs and MAEs of PIELM and BPIELM. Max-AEs and MAEs of PIELM are much

worse than BPIELM at the same sensor number, especially for the scenario with the small sensor number. In particular, the last two columns show predictions of PIELM and BPIELM with 19 sensors. The Max-AE of PIELM is extremely large, but BPIELM reduces the Max-AE to be less than 0.4. In a word, BPIELM outperforms PIELM at the same sensor number in terms of Max-AEs and MAEs.

To further study the impact of the different neuron number on PIELM and BPIELM, we choose three scenarios with $N_{bc} = 16, 28, 37$. The results are shown in Fig. 11. Under the noise scenario, PIELM hardly achieves the best MAE when the neuron number is equal to or close to the number of constraints, i.e., $N = N_f + N_b$. Obviously, the solution accuracy of PIELM is sensitive to the neuron number, while BPIELM gives much better predictions when the neuron number is larger enough. Moreover, as shown in the figure (from left to right), when the number of noisy sensors increases, MAEs of PIELM and BPIELM decreases and the fluctuation amplitude of the curve increases. However, the fluctuation amplitude of the curve using BPIELM is much smaller than that using PIELM.

Table 3 presents further comparisons of PINN, PIELM and BPIELM with regard to the accuracy and computational cost under three noise scales. In terms of MAEs, BPIELM is much more accu-

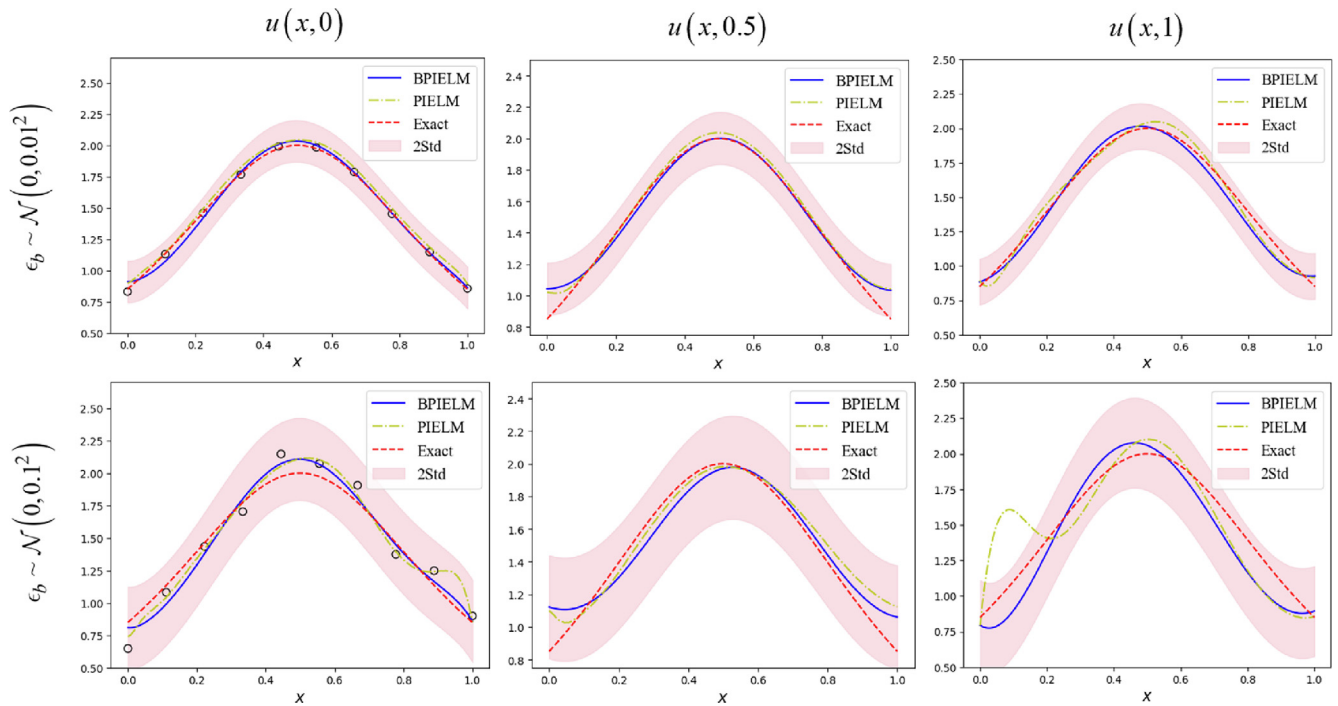


Fig. 8. Advection equation: The columns (from left to right) represent comparisons between BPIELM and PIELM at $t = 0, t = 0.5$ and $t = 1$, where $N = 150, N_f = 400$ and $N_b = 28$ are used. The rows (from top to down) represent two noise scales on the boundary, $\epsilon_b \sim \mathcal{N}(0, 0.01^2)$ and $\epsilon_b \sim \mathcal{N}(0, 0.1^2)$.

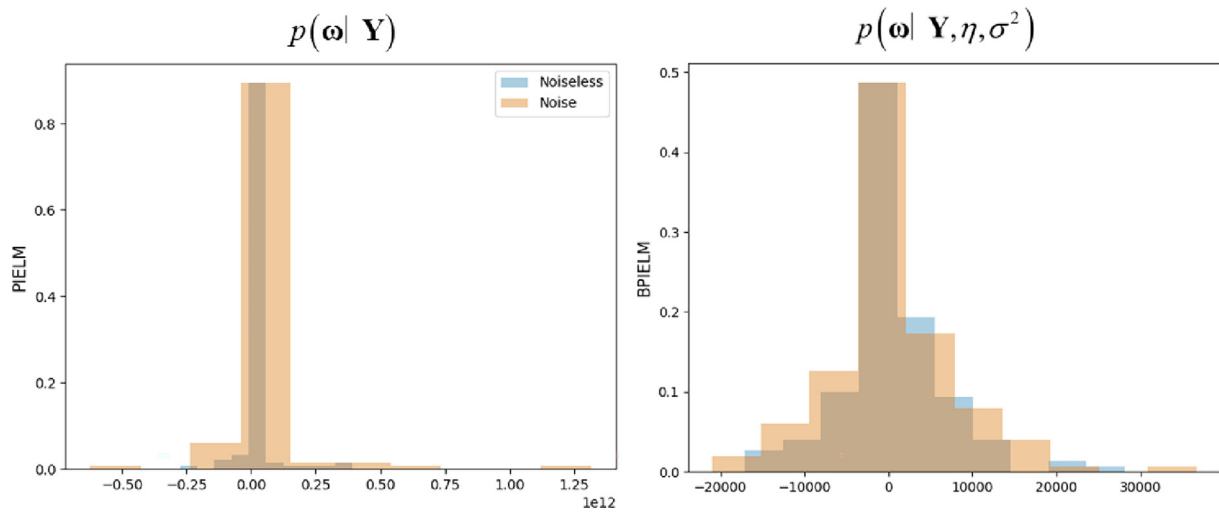


Fig. 9. Advection equation: The rows (from left to right) represent $p(\omega|\mathbf{Y})$ by PIELM and $p(\omega|\mathbf{Y}, \eta, \sigma^2)$ by BPIELM. The noise scale is $\epsilon_b \sim \mathcal{N}(0, 0.1^2)$. The number of neurons and training points is $N = 150, N_f = 400$ and $N_b = 28$, respectively..

rate than PIELM and PINN, while Max-AEs of BPIELM are close to PINN. For PINN, PIELM and BPIELM, Max-AEs and MAEs increase as the noise scale increase. Besides, PIELM and BPIELM are dramatically faster to train PINN (by two orders of magnitude).

4.3. Diffusion equation

The diffusion equation in one and two spatial dimensions (plus time) is given by

$$\begin{aligned} \frac{\partial u}{\partial t} - \nu \frac{\partial^2 u}{\partial x^2} &= f(x, t), (x, t) \in \Omega, \\ u(x_1, t) &= b_1(t), u(x_2, t) = b_2(t), \\ u(x, 0) &= h(x), \end{aligned} \quad (29)$$

where $f(x, t)$ is a source term, the constant $\nu = 0.01$ is the diffusion coefficient, and $\Omega = \{(x, t) | x \in [0, 1], t \in [0, 2]\}$. The expression for $f(x, t)$ is constructed by the following exact solution,

$$u(x, t) = \left[2 \cos\left(\pi x + \frac{\pi}{5}\right) + \frac{3}{2} \cos\left(2\pi x - \frac{3\pi}{5}\right) \right] \left[2 \cos\left(\pi t + \frac{\pi}{5}\right) + \frac{3}{2} \cos\left(2\pi t - \frac{3\pi}{5}\right) \right]. \quad (30)$$

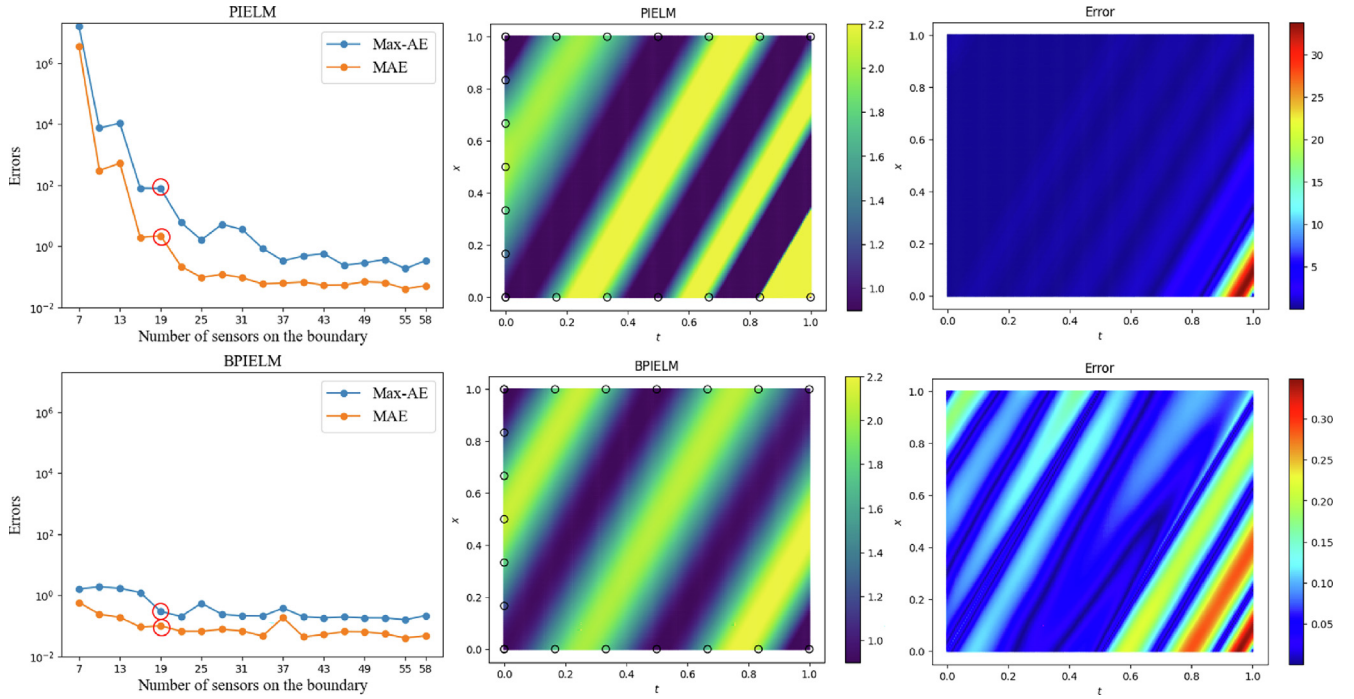


Fig. 10. Advection equation: The first column represents Max-AEs and MAEs under different number of sensors on the boundary, where $\epsilon_b \sim \mathcal{N}(0, 0.1^2)$, $N = 150$ and $N_f = 400$ are used. The second and third columns represent the prediction at $N_b = 19$ and its absolute error. The rows (from top to down) represent results of PIELM and BPIELM. Black circles represent the positions of sensors.

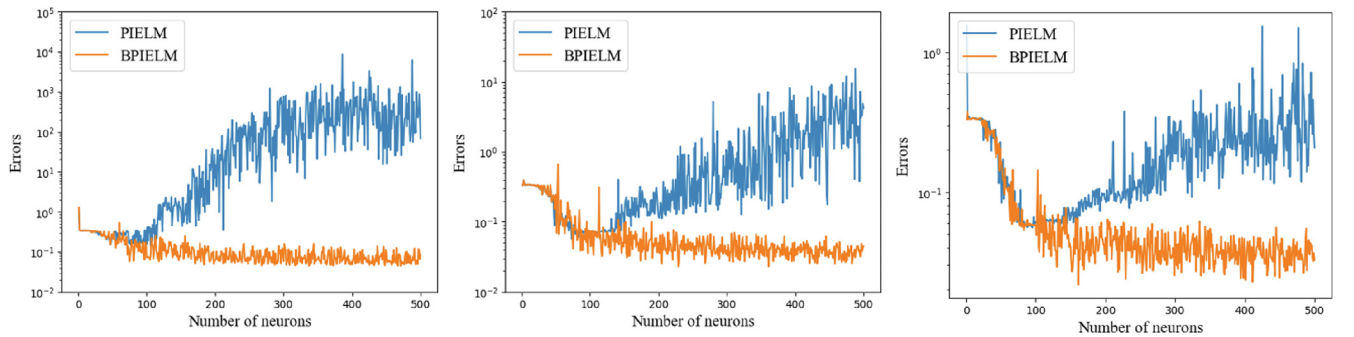


Fig. 11. Advection equation: The columns (from left to right) represent comparisons of MAEs between BPIELM and PIELM at $N_{bc} = 16, N_{bc} = 28$ and $N_{bc} = 37$, where $\epsilon_b \sim \mathcal{N}(0, 0.1^2)$ and $N_f = 400$ are used.

Table 3

Advection equation: The comparisons of PINN, PIELM and BPIELM under three noise scales. The number of training points is $N_f = 400$, $N_b = 28$ and the number of neurons is $N = 150$. PINN has four hidden layers with 100 neurons in each layer and the epoch is 4,000.

Noise scale	0.01			0.05			0.1		
	Max-AE	MAE	Time/s	Max-AE	MAE	Time/s	Max-AE	MAE	Time/s
PINN	0.162	0.033	153	0.183	0.055	151	0.223	0.082	153
PIELM	0.442	0.037	0.98	0.541	0.047	1.01	0.670	0.070	1.22
BPIELM	0.170	0.027	1.12	0.209	0.039	1.18	0.224	0.066	1.30

Assuming that boundary conditions ($b_1(t), b_2(t)$ and $h(x)$) are unknown, we consider the Gaussian noise in measurements and only have N_b sensors distributed on the boundary equidistantly.

We approximate the solution using BPIELM to solve the diffusion equation. Concretely, governing equations and measurements on the boundary are encoded into the linear system, and then the Bayesian method is used to estimate output weights. In this test, the number of neurons is $N = 180$ and the number of training points is $N_f = 400$ and $N_b = 28$. The noise scale on the boundary

is $\epsilon_b \sim \mathcal{N}(0, 0.1^2)$. Fig. 12 illustrates the results of PIELM and BPIELM, where the data noise scale on the boundary is $\epsilon_b \sim \mathcal{N}(0, 0.1^2)$. BPIELM makes more accurate predictions for the solution than PIELM and gives uncertainty quantification for the solution. More concretely, MAEs of PIELM and BPIELM are 0.0787 and 0.0186, respectively. The absolute errors are concentrated over the area near $t = 0$ but also spread all over the spatial-temporal domain. Similarly, for BPIELM, the area near $t = 0$ has a relatively

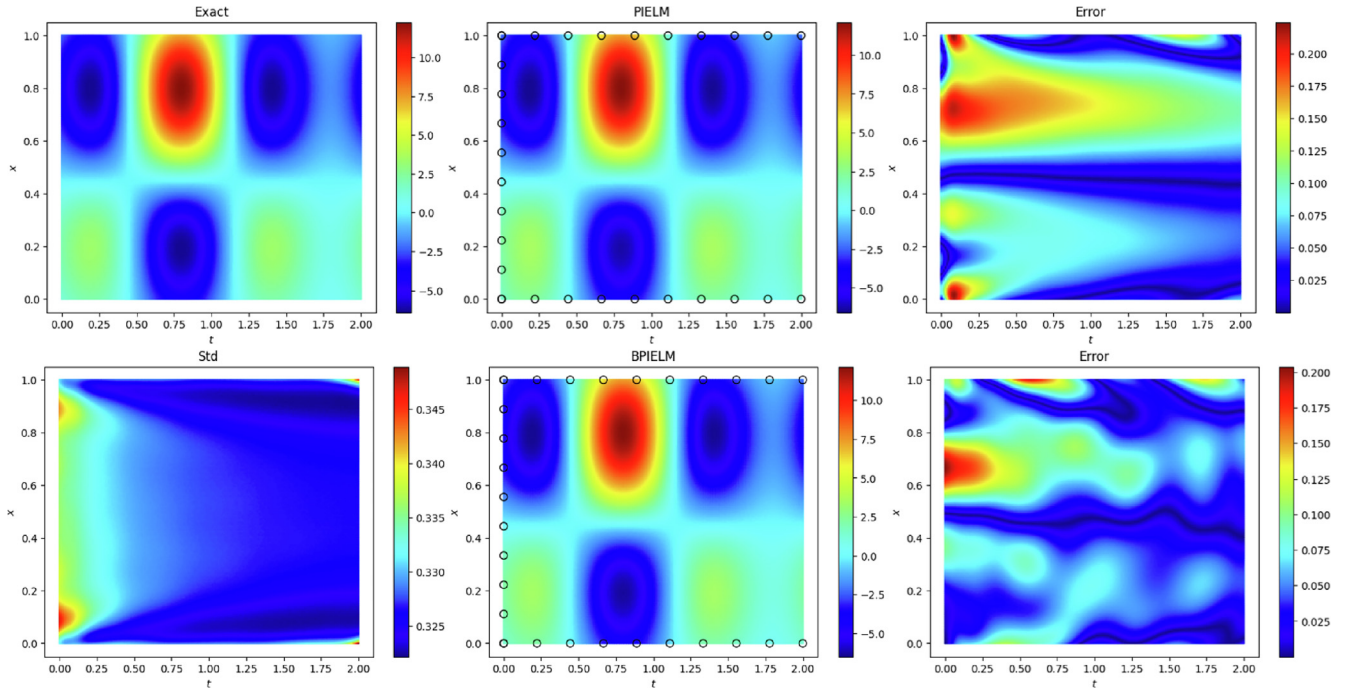


Fig. 12. Diffusion equation: The first row represents the exact solution, the PIELM solution, and its absolute error, respectively. The second row represents the standard deviations for u using BPIELM, the BPIELM solution, and its absolute error, respectively. The data noise scales on the boundary is $\epsilon_b \sim \mathcal{N}(0, 0.1^2)$, the number of neurons is $N = 180$ and the number of training points is $N_f = 400$ and $N_b = 28$. Black circles represent positions of sensors.

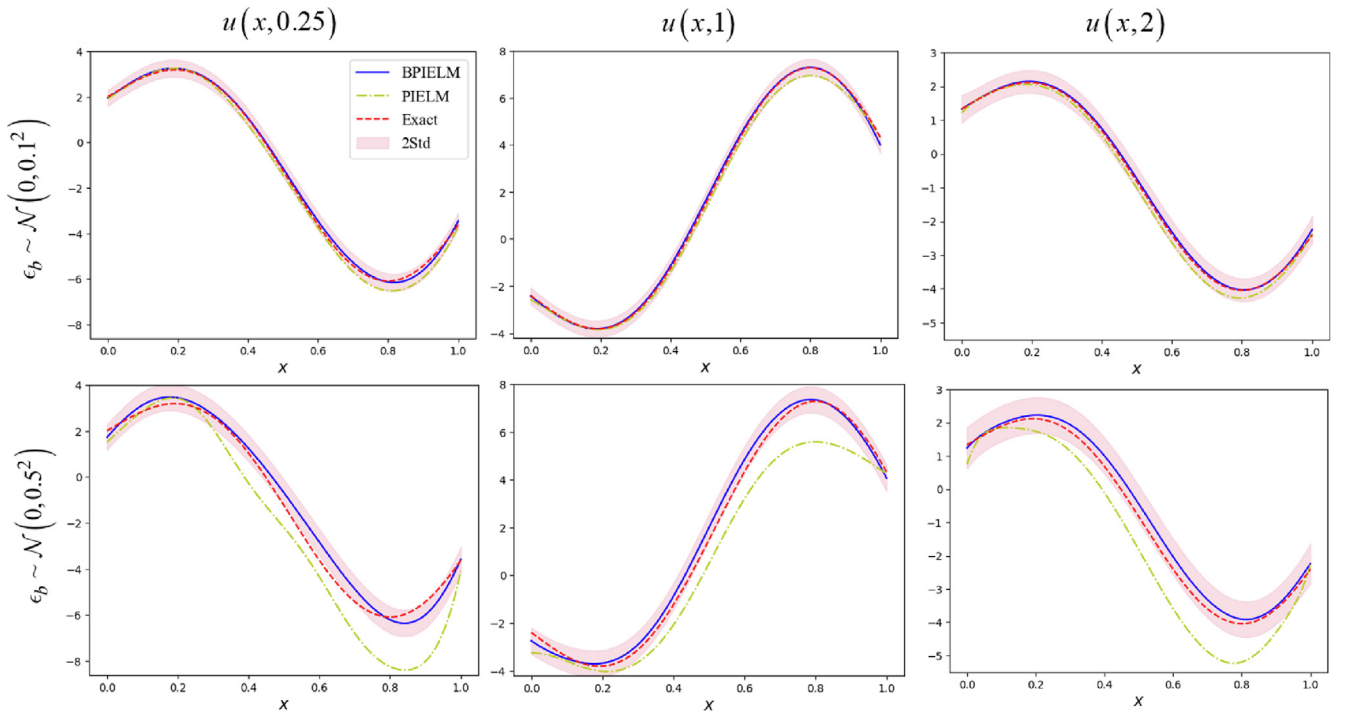


Fig. 13. Diffusion equation: The columns (from left to right) represent the comparisons between BPIELM and PIELM at $t = 0.25, t = 1$ and $t = 2$, where $N = 180, N_f = 400$ and $N_b = 30$ are used. The rows (from top to down) represent two noise scales on the boundary, $\epsilon_b \sim \mathcal{N}(0, 0.1^2)$ and $\epsilon_b \sim \mathcal{N}(0, 0.5^2)$.

large standard deviation. To compare two different noise scales, i.e., $\epsilon_b \sim \mathcal{N}(0, 0.1^2)$ and $\epsilon_b \sim \mathcal{N}(0, 0.5^2)$, Fig. 13 provides the results between PIELM and BPIELM at three temporal snapshots $t = 0.25, 1, 2$. The errors of BPIELM are mostly bounded by two standard deviations and the errors between predictions and the exact solution increase with the increasing noise scale. In particu-

lar, when it comes to the large noise scale, the performance of PIELM is much worse than BPIELM. The distributions of output weights under noise and noiseless scenarios are illustrated in Fig. 14. BPIELM uses the Bayesian method to learn the posterior distributions $p(\omega|\mathbf{Y}, \eta, \sigma^2)$, which are close under noise and noiseless scenarios. However, the distributions of parameters $p(\omega|\mathbf{Y})$

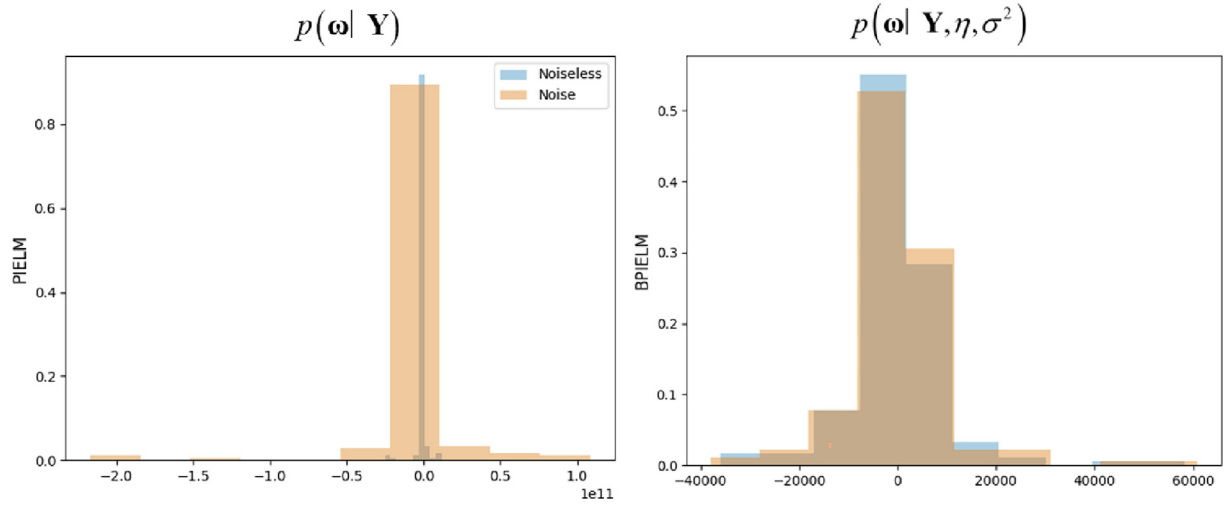


Fig. 14. Diffusion equation: The rows (from left to right) represent $p(\omega|Y)$ by PIELM and $p(\omega|Y, \eta, \sigma^2)$ by BPIELM. The noise scale is $\epsilon_b \sim \mathcal{N}(0, 0.1^2)$. The number of neurons and training points are $N = 150$, $N_f = 400$ and $N_b = 28$, respectively..

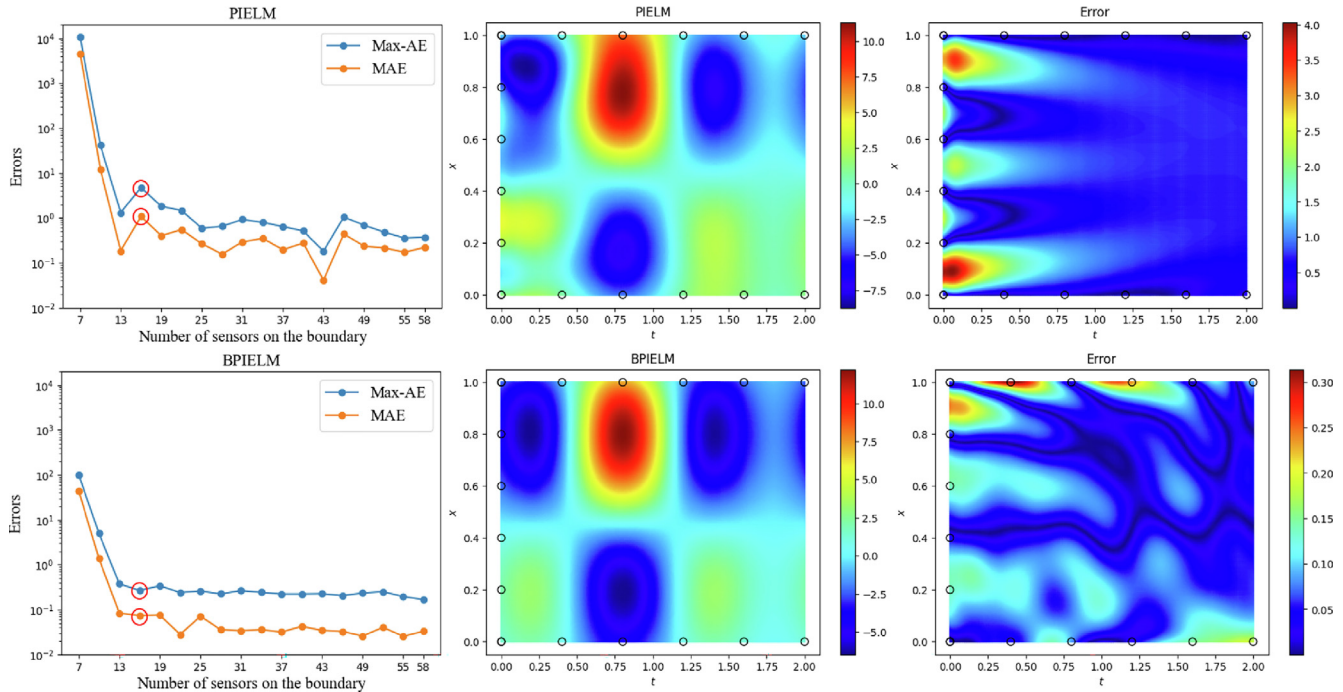


Fig. 15. Diffusion equation: The first column represents Max-AEs and MAEs under different number of sensors on the boundary, where $\epsilon_b \sim \mathcal{N}(0, 0.1^2)$, $n = 180$ and $N_f = 400$ are used. The second and third columns represent predictions at $N_b = 16$ and its absolute error. The rows (from top to down) represent the results of PIELM and BPIELM. Black circles represent the positions of sensors.

learned by PIELM are quite different under noise and noiseless scenarios.

To study the effect of the different sensor number for PIELM and BPIELM, we compare the results in Fig. 15. As the first column shows, BPIELM outperforms PIELM in terms of Max-AEs and MAEs under the same sensor number. Max-AEs and MAEs of PIELM and BPIELM are both worse when the sensor number is extremely small, while MAEs of BPIELM tend to be less than 10^{-1} when the sensor number is more than 13. In particular, the results of PIELM and BPIELM with 16 sensors are shown in the last two columns, where BPIELM provides much more predictions than PIELM.

Fig. 16 illustrates the effect of the different neuron number with a fixed number of training points in PIELM and BPIELM on the predictive accuracy. As the figure shows, the MAE curve of PIELM is growing up when $N > 200$, while the MAE curve of BPIELM converges to a small value and fluctuates in a small range when the

neuron number is large enough. It indicates that the solution accuracy of PIELM is more sensitive to the neuron number than that of BPIELM.

In Table 4, we provide further comparisons of PINN, PIELM and BPIELM in terms of the accuracy and the computational cost under three noise scales. The computational cost of BPIELM and PIELM is comparable, while BPIELM is more accurate than PIELM. Besides, compared to PINN, the training time of BPIELM and PIELM has been reduced by two orders of magnitude.

4.4. Inverse problem

In this subsection, we consider quickly identifying unknown problem parameters in the PDEs by BPIELM. In current research works, PIELM has not been extended to inverse PDE problems, but PIELM can also consider problem parameters as output weights

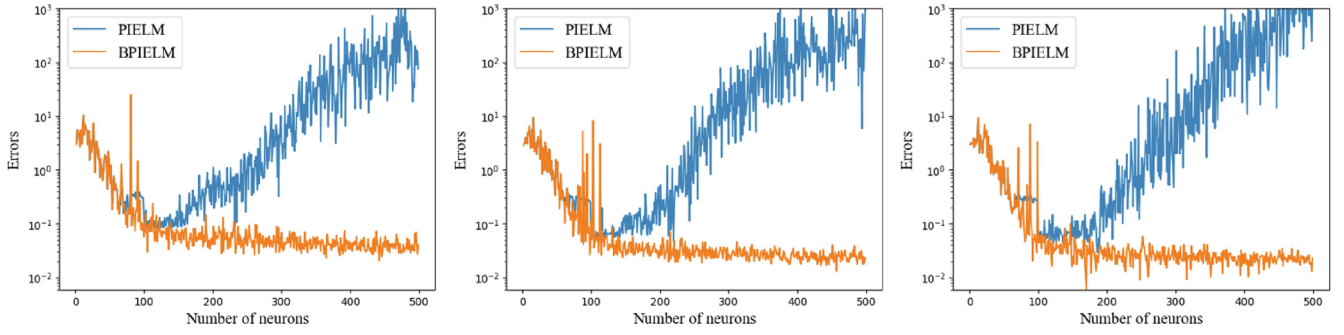


Fig. 16. Diffusion equation: The columns (from left to right) represent the MAE comparisons between BPIELM and PIELM at $N_{bc} = 42, N_{bc} = 62$ and $N_{bc} = 92$, where $\epsilon_b \sim \mathcal{N}(0, 0.1^2)$ and $N_f = 400$ are used.

Table 4

Diffusion equation: The comparisons of PINN, PIELM and BPIELM under three noise scales. The number of training points is $N_f = 400$, $N_b = 28$ and the number of neurons is $N = 180$. PINN has four hidden layers with 100 neurons in each layer and the epoch is 4,000.

Noise scale	0.01			0.05			0.1		
	Max-AE	MAE	Time/s	Max-AE	MAE	Time/s	Max-AE	MAE	Time/s
PINN	0.082	0.019	181	0.138	0.032	182	0.182	0.033	182
PIELM	0.163	0.025	0.84	0.179	0.040	0.93	0.226	0.079	0.91
BPIELM	0.033	0.007	1.67	0.115	0.021	1.76	0.118	0.019	1.68

like BPIELM and then uses Moore–Penrose generalized inverse to solve output weights for inverse PDE problems.

4.4.1. Poisson equation

As the first inverse problem test, we consider the following linear Poisson equation,

$$\begin{aligned} u_{xx} &= -\lambda_1 \cdot \sin(0.7x) - \lambda_2 \cdot \cos(1.5x), \quad x \in \Omega, \\ u(x_1) &= b_1, \quad u(x_2) = b_2, \end{aligned} \quad (31)$$

where problem parameters (λ_1 and λ_2) are unknown, x_1 and x_2 are boundary points, and $\Omega = \{x|x \in [-10, 10]\}$. The exact solution is considered in this test,

$$u = \sin(0.7x) + \cos(1.5x) - 0.1x. \quad (32)$$

Assuming that the exact solution is unknown, we have N_u sensors for u , which are equidistantly distributed in Ω . Besides, two sensors are placed at $x = -10$ and $x = 10$ to provide the boundary conditions. The Gaussian noises in the measurements are considered, i.e., ϵ_u and ϵ_b .

Let us compare BPIELM with PIELM and PINN for solving the Poisson equation. Fig. 17 compares the solution using BPIELM, PIELM and PINN under two data noise scales. In the setting of PINN, the NN has three hidden layers with 50 neurons in each layer and the tanh activation function is used. The training epoch with Adam

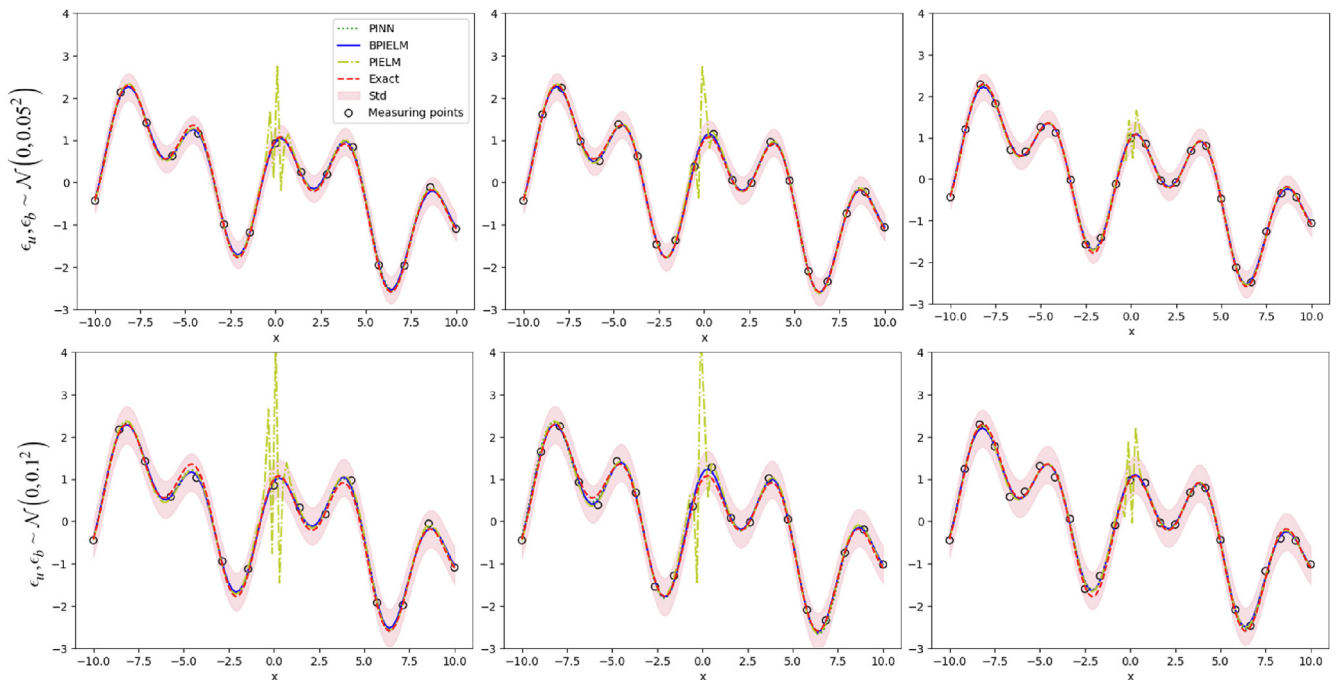


Fig. 17. Poisson equation – inverse problem: The comparisons of PINN, PIELM, and BPIELM with $N_u = 13, 18, 23$ under two noise scales. The number of neurons is $N = 100$ and the number of training points is $N_f = 100$ and $N_b = 2$. Black circles represent the positions of sensors.

is 4,000. For PIELM and BPIELM, the neuron number is $N = 100$ and the number of training points is $N_f = 100$ and $N_b = 2$. As shown in the figure, BPIELM gives uncertainty and provides accurate predictions. The errors of BPIELM are mostly bounded by two standard deviations, which increase as the noise scale increases. Besides, the predictions of PINN are close to BPIELM, which are better than PIELM. Under noise scenarios, PIELM using Moore–Penrose generalized inverse with limited sensors is prone to overfit noisy data, causing worse predictions. The columns from left to right in the figure represent the predictions with $N_u = 13, 18, 23$. The predictions with more sensors are better than those with fewer sensors. Besides, the larger noise scale in the data also causes worse predictions. Fig. 18 compares the distributions of output weights learned by PIELM and BPIELM under noise and noiseless scenarios. The posterior distributions $p(\omega|\mathbf{Y}, \eta, \sigma^2)$ learned by BPIELM are close

under noise and noiseless scenarios, but $p(\omega|\mathbf{Y})$ learned by PIELM are quite different.

Table 5 provides further comparison with PIELM, BPIELM and PINN for both cases with noise scale, i.e., $\epsilon_u, \epsilon_b \sim \mathcal{N}(0, 0.05^2)$ and $\epsilon_u, \epsilon_b \sim \mathcal{N}(0, 0.1^2)$. The data in the table shows that BPIELM is much more accurate than PIELM and is dramatically faster to train PINN (by two orders of magnitude). BPIELM and PINN give close predictions. Besides, we observe that the larger noise scale leads to worse predictions for the problem parameters and the solution. In a word, the results show the effectiveness of BPIELM in identifying the unknown problem parameters and solving the solution from sparse noisy data.

Fig. 19 demonstrates the effect of the different neuron number in PIELM and BPIELM on identifying problem parameters and pre-

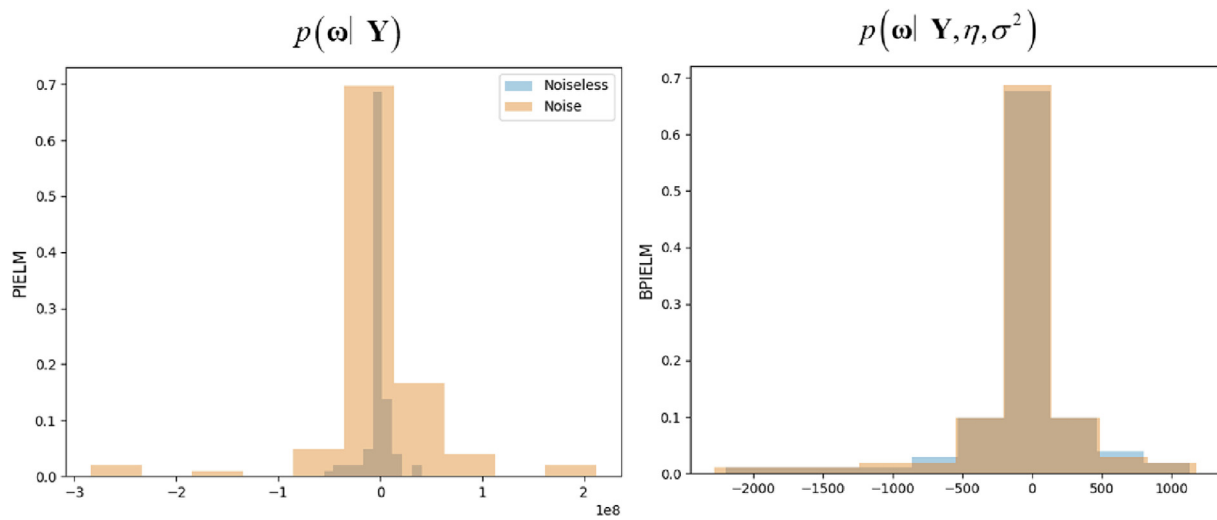


Fig. 18. Poisson equation – inverse problem: The rows (from left to right) represent $p(\omega|\mathbf{Y})$ by PIELM and $p(\omega|\mathbf{Y}, \eta, \sigma^2)$ by BPIELM. The noise scale is $\epsilon_b \sim \mathcal{N}(0, 0.05^2)$. The number of neurons and training points is $N = 100, N_f = 100$ and $N_b = 2$, respectively..

Table 5

1D Poisson equation - inverse problem: The comparisons of PINN, PIELM and BPIELM under two noise scales. The number of neurons is $N = 100$ and the number of training points is $N_f = 100, N_u = 18, N_b = 2$. The exact solutions for λ_1 and λ_2 are 0.49 and 2.25, respectively.

Method	$\epsilon_u, \epsilon_b \in 0.05$					$\epsilon_u, \epsilon_b \in 0.1$				
	λ_1	λ_2	Max-AE	MAE	Time/s	λ_1	λ_2	Max-AE	MAE	Time/s
PINN	0.491	2.307	0.073	0.027	225.30	0.488	2.370	0.160	0.068	240.12
PIELM	0.484	2.390	1.808	0.081	0.71	0.477	2.280	3.619	0.168	0.77
BPIELM	0.490	2.300	0.076	0.027	2.51	0.480	2.300	0.155	0.054	2.53

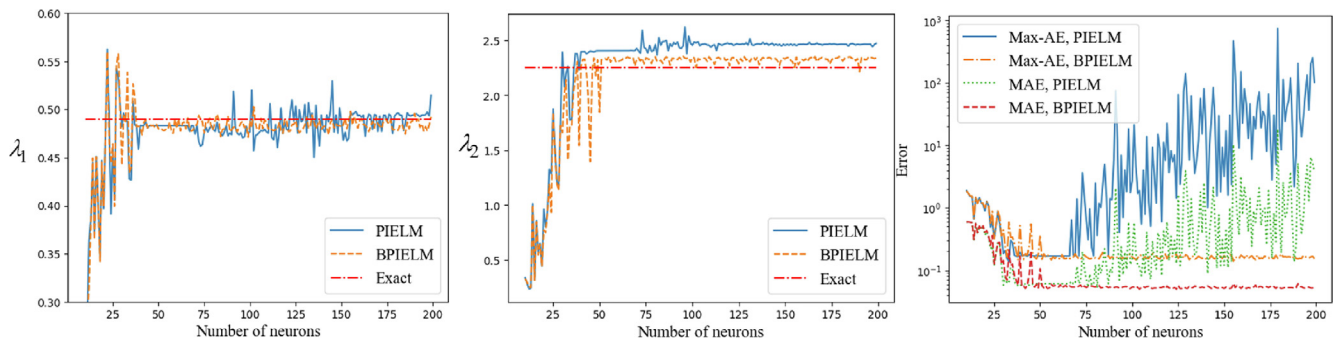


Fig. 19. 1D Poisson equation – inverse problem: The comparisons of PIELM and BPIELM under the different neuron number. The first two columns represent the predictions for the system parameters, λ_1 and λ_2 , where the exact solutions for λ_1 and λ_2 are 0.49 and 2.25. The last column represents the comparisons of Max-AEs and MAEs. The sensor number is $N_u = 18$, the neuron number is $N = 100$ and the number of training points is $N_f = 100$ and $N_b = 2$.

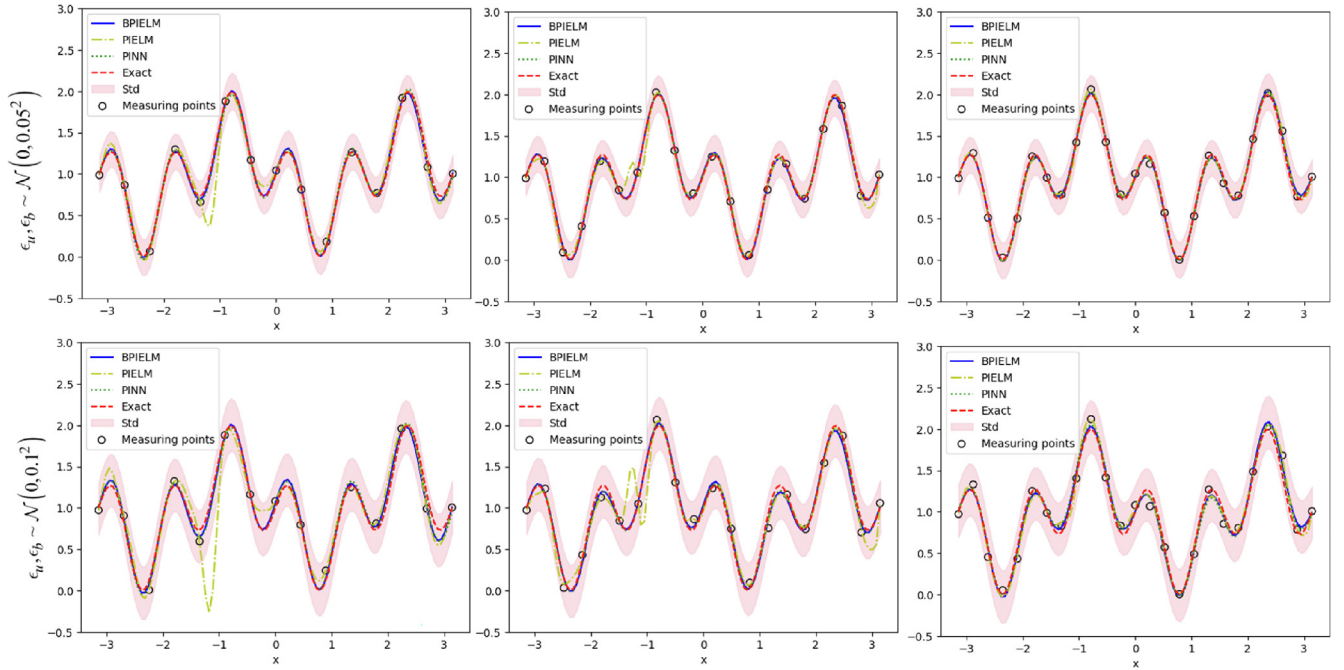


Fig. 20. Helmholtz equation – inverse problem: The comparisons of PINN, PIELM and BPIELM with $N_u = 13, 18, 23$ under two noise scales. The neuron number is $N = 100$ and the number of training points is $N_f = 100$ and $N_b = 2$. Black circles represent the positions of sensors.

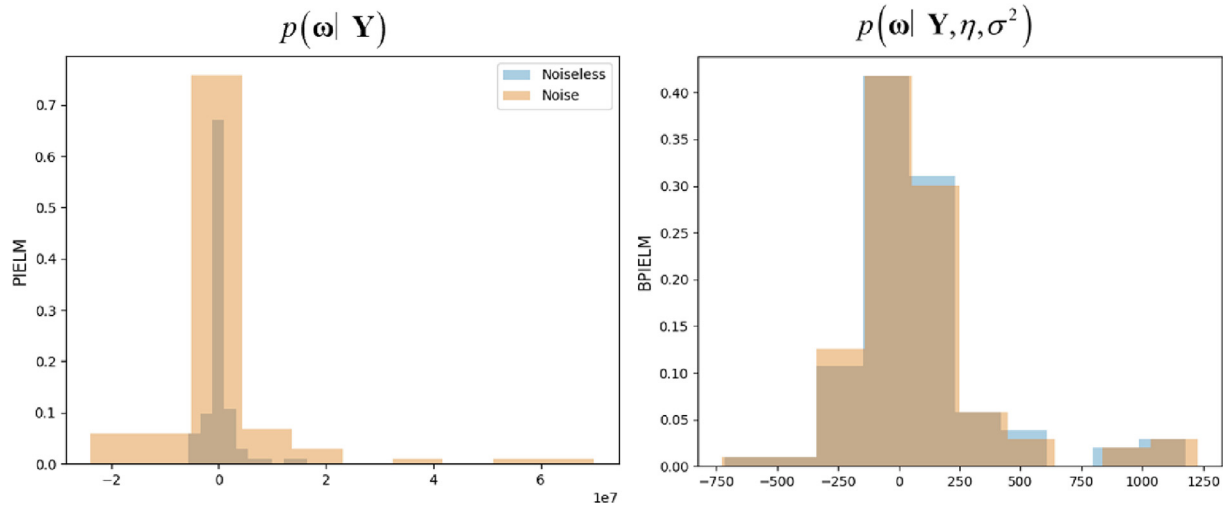


Fig. 21. Helmholtz equation – inverse problem: The rows (from left to right) represent $p(\omega|Y)$ by PIELM and $p(\omega|Y, \eta, \sigma^2)$ by BPIELM. The noise scale is $\epsilon_b \sim \mathcal{N}(0, 0.01^2)$. The number of neurons and training points is $N = 100, N_f = 100$ and $N_b = 2$, respectively.

dicting the solution. The data in the first two columns show that BPIELM is more stable for the neuron number, while PIELM is more sensitive to the neuron number. Note that when the neuron number is large enough, the predictive problem parameters of BPIELM converge to exact values and fluctuate in a small range. As the last column shows, Max-AEs as well as MAEs of PIELM and BPIELM are close under the small neuron number and then PIELM achieves the best Max-AEs and MAEs within a certain neuron number, but it is difficult for PIELM to find appropriate neuron number in practice except for the experiments. However, BPIELM alleviates overfitting noisy data and a large enough number of neurons is chosen for BPIELM, which is more appropriate for noise scenarios.

4.4.2. Helmholtz equation

In the next test, we consider the following 1D linear Helmholtz equation

$$\begin{aligned} u_{xx} + k^2 u &= -\lambda_1 f_1(x) + \lambda_2 \cdot f_2(x) - \lambda_3, x \in \Omega, \\ u(x_1) &= b_1, \quad u(x_2) = b_2, \end{aligned} \quad (33)$$

where problem parameters (λ_1, λ_2 and λ_3) are unknown, the wave number is $k^2 = 10, x_1$ as well as x_2 are boundary points, $f_1(x) = \sin(2x) \cos(4x), f_2(x) = \cos(2x) \sin(4x)$, and $\Omega = \{x|x \in [-2\pi, 2\pi]\}$. The exact solution is given by,

$$u = \sin(2x) \cos(4x) + 1. \quad (34)$$

Assuming that the exact solution is unknown, we have N_u sensors for u , which are equidistantly distributed in Ω . Besides, two sensors are placed at $x_1 = -2\pi$ and $x_2 = 2\pi$ to provide the boundary conditions for u . The Gaussian noises in the measurements are considered, i.e., ϵ_u and ϵ_b .

We next compare BPIELM with PIELM and PINN for solving the Helmholtz equation, where more unknown problem parameters are identified. Fig. 20 compares the solution using BPIELM, PIELM, and PINN. In the setting of PINN, we use four hidden layers with 100 neurons in each layer and use 4,000 epochs to train the NN with the tanh activation function. For PIELM and BPIELM, the neuron number is $N = 100$. As the figure shows, BPIELM over PIELM and PINN has a twofold benefit: (1) BPIELM gives uncertainty for the solution, and errors are mostly bounded by two standard deviations. (2) BPIELM provides more accurate predictions than PIELM, and BPIELM reduces the computational cost to achieve close or better predictions than PINN. Fig. 21 illustrates distributions of output

weights learned by PIELM and BPIELM under noise and noiseless scenarios. Obviously, distributions of output weights $p(\omega|\mathbf{Y})$ learned by PIELM are quite different under noiseless scenarios, but posterior distributions $p(\omega|\mathbf{Y}, \eta, \sigma^2)$ learned by BPIELM are close.

Table 6 provides further comparisons with PIELM, BPIELM, and PINN for both cases with two noise scales. The results show the effectiveness of BPIELM in identifying the unknown problem parameters and solving the solution from the sparse noisy data. BPIELM is much more accurate than PIELM and considerably cheaper than PINN in terms of computational cost. The training time with the BPIELM is on the order of two seconds. In contrast, it takes 400 s to train PINN to obtain close predictions.

Fig. 22 illustrates the effect of different neuron numbers in PIELM and BPIELM on identifying problem parameters. The results show that PIELM is dramatically more sensitive to the neuron number than BPIELM. For BPIELM, when the neuron number is

Table 6

1D Helmholtz equation – inverse problem: The comparisons of PINN, PIELM and BPIELM under two noise scales. The neuron number is $N = 100$ and the number of training points is $N_f = 100$, $N_b = 2$. The exact solutions for λ_1 , λ_2 and λ_3 are 10, 16, -10 , respectively.

Method	$\epsilon_u, \epsilon_b \in 0.05$						$\epsilon_u, \epsilon_b \in 0.1$					
	λ_1	λ_2	λ_3	Max-AE	MAE	Time/s	λ_1	λ_2	λ_3	Max-AE	MAE	Time/s
PINN	9.84	15.84	-9.96	0.034	0.015	399.38	9.71	15.71	-9.93	0.071	0.031	398.13
PIELM	9.71	14.8	-10	0.337	0.047	0.69	9.41	13.5	-10	0.683	0.096	0.81
BPIELM	9.80	16.0	-10	0.051	0.018	2.59	9.60	15.9	-10	0.091	0.035	2.51

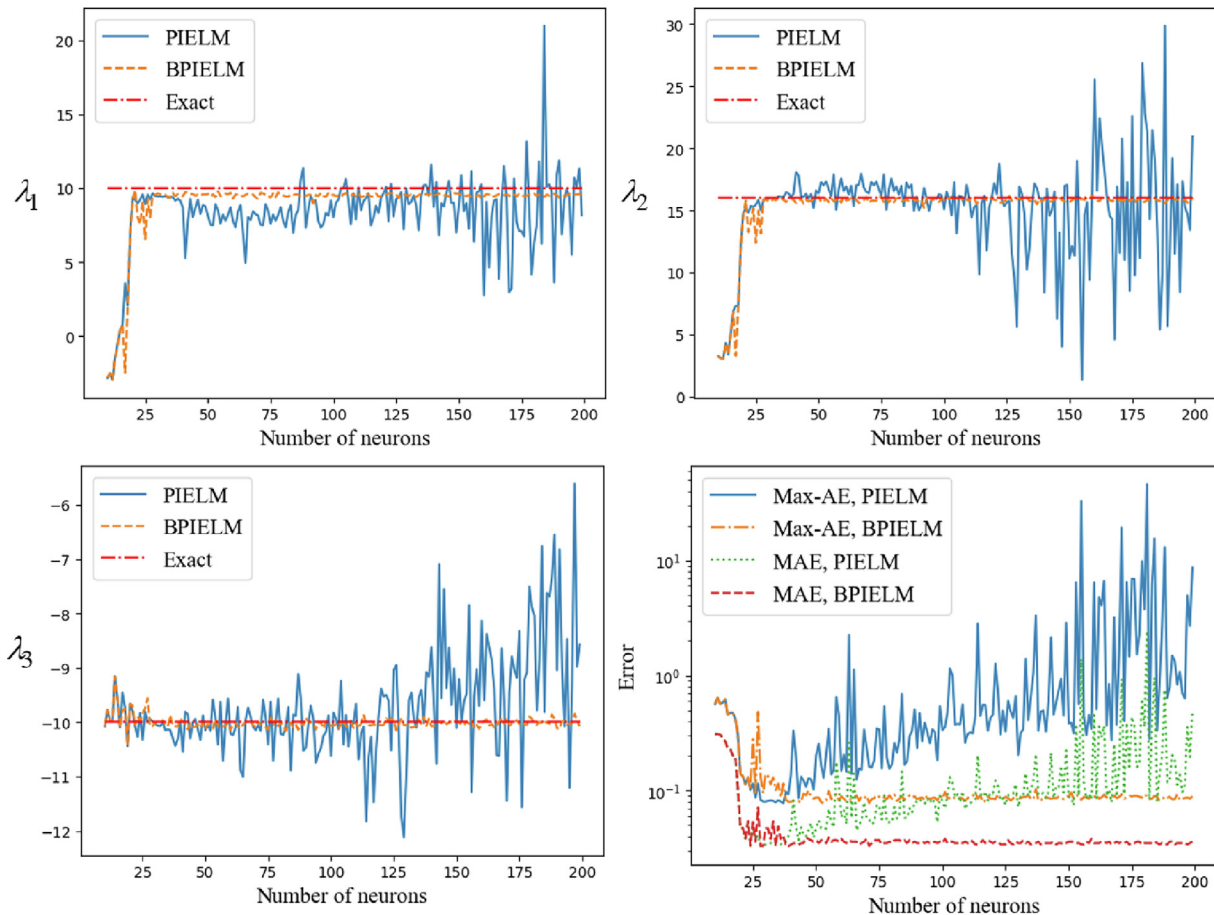


Fig. 22. 1D Helmholtz equation – inverse problem: The comparisons of PIELM and BPIELM under the different neuron number. The first row represents predictions for the system parameters, λ_1 and λ_2 . The second row represents predictions for the system parameter λ_3 and comparisons of Max-AEs and MAEs, where $N = 100$, $N_f = 100$, and $N_b = 2$ are used. The exact solutions for λ_1 , λ_2 and λ_3 are 10, 16, -10 , respectively.

large enough, predictive problem parameters converge to exact values and fluctuate in a small range, which indicates that BPIELM avoids overfitting noisy data. For Max-AE and MAE curves, under the small neuron number, MAEs between PIELM and BPIELM are close and worse. It means that a shallow NN architecture both for PIELM and BPIELM can not learn enough physics information to depict the whole solution. As the neuron number increases, MAEs of PIELM increase after reaching the minimum value in a certain range, while the Max-AE and MAE curves of BPIELM converge to the values and fluctuate in a small range. In summary, considering the accuracy as well as the sensibility to the neuron number, BPIELM is a better approach than PIELM for noise scenarios.

5. Conclusion

In this work, we propose the BPIELM framework to quantify uncertainty arising from noisy data for forward and inverse PDE problems. The physical laws (PDE, boundary conditions, and measurements) are first incorporated into the NN as the cost function. Then a prior probability distribution is introduced in output weights and the Bayesian method is used to estimate the posterior. To support our claim, we examine BPIELM for forward and inverse PDE problems, including Poisson, advection, diffusion, and Helmholtz equations. In those tests under noise scenarios, the results show that BPIELM has a threefold benefit: (1) BPIELM provides uncertainty quantification for the solution and more accurate predictions for solutions as well as problem parameters than PIELM. (2) PIELM is prone to be drastically sensitive to the neuron number, but BPIELM alleviates the sensitivity characteristic and improves the robustness of the NN. (3) BPIELM is considerably cheaper than PINN in terms of computational cost.

Although BPIELM has exhibited some superior potential for designing fast and accurate scientific machine learning techniques, some things need to be resolved. The high efficiency of PIELM benefits from the linear nature of the final problem, therefore BPIELM, which inherits some characteristics of PIELM, is also limited to studying linear problems. How to further extend and improve this method is an interesting problem, and will be explored in future works.

CRedit authorship contribution statement

Xu Liu: Software, Methodology, Formal analysis, Writing – original draft. **Wen Yao:** Supervision, Funding acquisition, Project administration. **Wei Peng:** Conceptualization, Supervision. **Wei Zhou:** Software, Supervision, Data curation, Visualization, Funding acquisition.

Data availability

Data will be made available on request.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

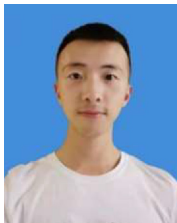
Acknowledgements

This work was supported by National Natural Science Foundation of China under Grant No.11725211, 52005505, and 62001502.

References

- [1] A.F. Psaros, X. Meng, Z. Zou, L. Guo, G.E. Karniadakis, Uncertainty quantification in scientific machine learning: Methods, metrics, and comparisons, arXiv preprint arXiv:2201.07766.
- [2] M. Alber, A. Buganza Tepole, W.R. Cannon, S. De, S. Dura-Bernal, K. Garikipati, G. Karniadakis, W.W. Lytton, P. Perdikaris, L. Petzold, et al., Integrating machine learning and multiscale modeling—perspectives, challenges, and opportunities in the biological, biomedical, and behavioral sciences, *NPJ Digital Med.* 2 (1) (2019) 1–11.
- [3] G.E. Karniadakis, I.G. Kevrekidis, L. Lu, P. Perdikaris, S. Wang, L. Yang, Physics-informed machine learning, *Nat. Rev. Phys.* 3 (6) (2021) 422–440.
- [4] M. Raissi, P. Perdikaris, G.E. Karniadakis, Physics informed deep learning (part i): Data-driven solutions of nonlinear partial differential equations, arXiv preprint arXiv:1711.10561.
- [5] M. Raissi, P. Perdikaris, G.E. Karniadakis, Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations, *J. Comput. Phys.* 378 (2019) 686–707.
- [6] S. Basir, I. Senocak, Physics and equality constrained artificial neural networks: Application to forward and inverse problems with multi-fidelity data fusion, arXiv preprint arXiv:2109.14860.
- [7] Z. He, F. Ni, W. Wang, J. Zhang, A physics-informed deep learning method for solving direct and inverse heat conduction problems of materials, *Mater. Today Commun.* 28 (2021).
- [8] X. Liu, W. Peng, Z. Gong, W. Zhou, W. Yao, Temperature field inversion of heat-source systems via physics-informed neural networks, *Eng. Appl. Artif. Intell.* 113 (2022).
- [9] X. Jin, S. Cai, H. Li, G.E. Karniadakis, Nsfnets (navier-stokes flow nets): Physics-informed neural networks for the incompressible navier-stokes equations, *J. Comput. Phys.* 426 (2021).
- [10] H.D. Kabir, A. Khosravi, M.A. Hosen, S. Nahavandi, Neural network-based uncertainty quantification: A survey of methodologies and applications, *IEEE Access* 6 (2018) 36218–36234.
- [11] X. Meng, H. Babaee, G.E. Karniadakis, Multi-fidelity bayesian neural networks: Algorithms and applications, *J. Comput. Phys.* 438 (2021).
- [12] Y. Zhu, N. Zabarar, P.-S. Koutsourelakis, P. Perdikaris, Physics-constrained deep learning for high-dimensional surrogate modeling and uncertainty quantification without labeled data, *J. Comput. Phys.* 394 (2019) 56–81.
- [13] Y. Yang, P. Perdikaris, Adversarial uncertainty quantification in physics-informed neural networks, *J. Comput. Phys.* 394 (2019) 136–152.
- [14] L. Yang, D. Zhang, G.E. Karniadakis, Physics-informed generative adversarial networks for stochastic differential equations, *SIAM J. Sci. Comput.* 42 (1) (2020) A292–A317.
- [15] A. Daw, M. Maruf, A. Karpatne, Pid-gan: A gan framework based on a physics-informed discriminator for uncertainty quantification with physics, in: *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*, 2021, pp. 237–247.
- [16] H. Wang, D.-Y. Yeung, A survey on bayesian deep learning, *ACM Computing Surveys (CSUR)* 53 (5) (2020) 1–37.
- [17] A.G. Wilson, P. Izmailov, Bayesian deep learning and a probabilistic perspective of generalization, *Adv. Neural Inform. Process. Syst.* 33 (2020) 4697–4708.
- [18] L. Yang, X. Meng, G.E. Karniadakis, B-pinns: Bayesian physics-informed neural networks for forward and inverse pde problems with noisy data, *J. Comput. Phys.* 425 (2021).
- [19] D. Zhang, L. Lu, L. Guo, G.E. Karniadakis, Quantifying total uncertainty in physics-informed neural networks for solving forward and inverse stochastic problems, *J. Comput. Phys.* 397 (2019).
- [20] J.N. Reddy, *Introduction to the finite element method*, McGraw-Hill Education, 2019.
- [21] R.J. LeVeque, *Finite difference methods for ordinary and partial differential equations: steady-state and time-dependent problems*, SIAM, 2007.
- [22] T. Bandai, T.A. Ghezzehei, Forward and inverse modeling of water flow in unsaturated soils with discontinuous hydraulic conductivities using physics-informed neural networks with domain decomposition, *Hydrol. Earth Syst. Sci. Discussions* (2022) 1–42.
- [23] Y. Yang, M. Hou, J. Luo, A novel improved extreme learning machine algorithm in solving ordinary differential equations by legendre neural network methods, *Adv. Difference Eqs.* 2018 (1) (2018) 1–24.
- [24] H. Sun, M. Hou, Y. Yang, T. Zhang, F. Weng, F. Han, Solving partial differential equation based on bernstein neural network and extreme learning machine algorithm, *Neural Process. Lett.* 50 (2) (2019) 1153–1172.
- [25] V. Dwivedi, B. Srinivasan, Physics informed extreme learning machine (pielm)—a rapid method for the numerical solution of partial differential equations, *Neurocomputing* 391 (2020) 96–118.
- [26] V. Dwivedi, N. Parashar, B. Srinivasan, Distributed learning machines for solving forward and inverse problems in partial differential equations, *Neurocomputing* 420 (2021) 299–316.
- [27] F. Calabrò, G. Fabiani, C. Siettos, Extreme learning machine collocation for the numerical solution of elliptic pdes with sharp gradients, *Comput. Methods Appl. Mech. Eng.* 387 (2021).
- [28] S. Dong, Z. Li, Local extreme learning machines and domain decomposition for solving linear and nonlinear partial differential equations, *Comput. Methods Appl. Mech. Eng.* 387 (2021).

- [29] E. Schiassi, R. Furfaro, C. Leake, M. De Florio, H. Johnston, D. Mortari, Extreme theory of functional connections: A fast physics-informed neural network method for solving ordinary and partial differential equations, *Neurocomputing* 457 (2021) 334–356.
- [30] D.T. Chang, Bayesian neural networks: Essentials, arXiv preprint arXiv:2106.13594.
- [31] N. Brosse, C. Riquelme, A. Martin, S. Gelly, É. Moulines, On last-layer algorithms for classification: Decoupling representation from uncertainty estimation, arXiv preprint arXiv:2001.08049.
- [32] J. Watson, J.A. Lin, P. Klink, J. Pajarinen, J. Peters, Latent derivative bayesian last layer networks, in: *International Conference on Artificial Intelligence and Statistics*, PMLR, 2021, pp. 1198–1206.
- [33] E. Soria-Olivas, J. Gomez-Sanchis, J.D. Martin, J. Vila-Frances, M. Martinez, J.R. Magdalena, A.J. Serrano, Belm: Bayesian extreme learning machine, *IEEE Trans. Neural Networks* 22 (3) (2011) 505–509.
- [34] Z. Jin, G. Zhou, D. Gao, Y. Zhang, Eeg classification using sparse bayesian extreme learning machine for brain–computer interface, *Neural Comput. Appl.* 32 (11) (2020) 6601–6609.
- [35] M.A. Bessa, P. Glowacki, M. Houlder, Bayesian machine learning in metamaterial design: Fragile becomes supercompressible, *Adv. Mater.* 31 (48) (2019) 1904845.
- [36] M. Evans, T. Swartz, *Approximating integrals via Monte Carlo and deterministic methods*, vol. 20, OUP Oxford, 2000.
- [37] L. Yang, X. Meng, G.E. Karniadakis, B-pinns: Bayesian physics-informed neural networks for forward and inverse pde problems with noisy data, *J. Comput. Phys.* 425. doi:10.1016/j.jcp.2020.109913.
- [38] J. Gawlikowski, C.R.N. Tassi, M. Ali, J. Lee, M. Humt, J. Feng, A. Kruspe, R. Triebel, P. Jung, R. Roscher, et al., A survey of uncertainty in deep neural networks, arXiv preprint arXiv:2107.03342.
- [39] T. Chen, E. Martin, Bayesian linear regression and variable selection for spectroscopic calibration, *Analytica chimica acta* 631 (1) (2009) 13–21.
- [40] D.J. MacKay, Probable networks and plausible predictions—a review of practical bayesian methods for supervised neural networks, *Network: Comput. Neural Syst.* 6(3) (1995) 469.
- [41] D.P. Kingma, J. Ba, Adam: A method for stochastic optimization, arXiv preprint arXiv:1412.6980.



Mr. Xu Liu received M.Sc. degree in Defense Innovation Institute, Chinese Academy of Military Science, Beijing, China. He is currently pursuing a Ph.D. in computer science and technology with the College of Defense Innovation Institute, Chinese Academy of Military Science, Beijing, China. His main research interests include physics-inspired machine learning and uncertainty-based optimization.



Dr. Wen Yao received the M.Sc. and Ph.D. degrees in aerospace engineering from National University of Defense Technology, Changsha, China, in 2007 and 2011, respectively. She is currently a Professor with the Defense Innovation Institute, Chinese Academy of Military Science, Beijing, China. Her research interests include spacecraft systems engineering, multidisciplinary design optimization, uncertainty-based optimization, and data-driven surrogate modeling and evolutionary optimization.



Dr. Wei Peng received the B.Sc. and PhD. degrees in mathematics from Fudan University, Shanghai, China in 2014 and Natinal University of Defense Technology, Changsha, China in 2019, respectively. He is currently an associate professor with Defense Innovation Institute, Chinese Academy of Military Science, Beijing, China. His research interests include evolutionary optimization, convex analysis, stochastic optimization and physics-inspired machine learning.



Dr. Weien Zhou received the B.Sc. degree in mathematics from Nanjing University, Nanjing, China, in 2012, and the Ph.D. degree in computational mathematics from the National University of Defense Technology, Changsha, China, in 2017. He is currently an Assistant Professor with the National Innovation Institute of Defense Technology, Chinese Academy of Military Science, Beijing, China. His current research interests include numerical methods for stochastic systems, uncertainty quantification, and large-scale optimization.